

Received 4 October 2025, accepted 6 November 2025,
date of publication 10 November 2025, date of current version 18 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3631153

APPLIED RESEARCH

Accelerating Probabilistic Planning Research: High-Speed Implementations of Stochastic Markov Decision Processes and Their Catalogization

ROBIN SCHMÖCKER¹, ALEXANDER DOCKHORN², (Member, IEEE),
AND BODO ROSENHAHN¹

¹Faculty of EECS, Leibniz University Hannover, 30167 Hannover, Germany

²SDU Metaverse Lab, University of Southern Denmark, 5230 Odense, Denmark

Corresponding author: Robin Schmöcker (schmoecker@tnt.uni-hannover.de)

ABSTRACT Probabilistic planning research, such as improvements to Monte Carlo Tree Search, is faced with two challenges. Firstly, one usually requires an algorithm's evaluation on a variety of test problems, trying several hundred or even thousands of parameter combinations, which in turn requires a large number of models that can be efficiently queried. Secondly, one has to identify a suitable set of problems to do the evaluation on. This work is made up of two components that together provide a solution for both problems for the probabilistic planning community. The first component of our work is software, which provides numerous models that are implemented by hand with domain-specific optimizations for optimal runtime. In particular, the software component contains 22 parametrized, non-deterministic Markov Decision Processes (MDP), which is currently the largest open-source, non-interpreter-based stochastic MDP library. The implementation provides access to methods required for planning (e.g., copying a state) as well as for reinforcement learning (e.g., getting an observation) through a C++ and a Python interface, which itself satisfies the Gymnasium interface. The second component of this work contains a detailed description as well as a statistical analysis of each model's properties, which acts as a catalogization of the environments, enabling researchers to quickly select a set of environments satisfying their specific criteria.

INDEX TERMS Artificial intelligence, sequential decision-making, benchmark, search.

NOMENCLATURE

MCTS	Monte Carlo Tree Search.
MDP	Markov Decision Process.
RDDL	Relational Dynamic Influence Diagram Language.
SG	Stochastic Game.

I. INTRODUCTION

Probabilistic planning research, such as applying automatic abstractions to MCTS [1], [2], [3], [4], [5], usually requires an algorithm's evaluation on a variety of test problems.

The associate editor coordinating the review of this manuscript and approving it for publication was Derek Abbott¹.

Furthermore, during development, one might conduct test runs on a few environments, but with several hundred or even thousands of parameter combinations. This brings two hurdles that our work aims to overcome.

The first hurdle is that problem models have to be queried efficiently to conduct numerous test runs. To solve this issue, researchers commonly either implement a few problems manually [4], [6] or use a slow interpreter for an array of test problems that are written in a high-level descriptive language such as RDDL [7] or a description language for strategy games such as Stratega [8]. We refer to the latter approach as interpreter-based.

The second hurdle is that the problem models to conduct research on have to be identified first. When designing a

new algorithm, it can also be time-consuming to identify suitable problems on which to evaluate the agent. One might require problems with a large action space, problems with a low stochastic branching factor, or grid-based navigation problems.

This document is structured as follows. Next, in Sections II and III, we present the main contributions of this article, which address the aforementioned two hurdles. In particular, we introduce a software framework that enables efficient execution of a large number of test runs across various fully-observable probabilistic planning tasks, overcoming the performance limitations of interpreter-based frameworks and the limited access to environments of existing manual implementations. The implementation accompanying this article has already enabled the experiments of existing work [2] and it would have been able to greatly expand the number of tested environments or parameters in previous works, which is a common limitation of research in automatic abstraction for MCTS [4], [5], [6], [9], [10]. The implementation is publicly available at: https://github.com/codebro634/Benchmark_Games.git.

Secondly, we provide a catalogization of all implemented planning tasks to streamline the identification of suitable benchmarks for evaluating planning algorithms. Afterwards, in Section IV, we compare the features of our framework with other planning frameworks. The article is concluded in Section V with a brief discussion of future work. The Appendix contains a precise definition of the formal framework that the implemented planning problems satisfy, namely, we will be defining finite MDPs and more generally SGs in the Appendix Section A. Then, in Appendix Section B, we provide descriptions of the implemented stochastic MDPs and in Appendix Section C a description of the two-player SGs, which together make up the catalogue that we will be referring to throughout this article.

II. A UNIFIED SOFTWARE FRAMEWORK FOR PROBABILISTIC PLANNING RESEARCH

To address the first hurdle, which is the need for efficiently queriable models, we manually implemented 33 planning tasks in a unified framework that we call *BenchmarkGames*. This implementation is publicly available at: https://github.com/codebro634/Benchmark_Games.git. The core of our implementation is 22 non-deterministic MDPs, which is one of the main subjects of interest for probabilistic planning research. As we plan to eventually expand our framework beyond probabilistic planning research, our implementation also includes the implementation of 11 two-player SGs. An MDP and SG definition is provided in the Appendix Section A. While this already presents a comprehensive benchmarking tool, the set of MDPs will be extended in future updates. We implemented each environment with a focus on efficiency in C++, employing domain-specific optimisations such as bit manipulation when suitable. Since we aim to make this as accessible as possible, we fully avoided using any non-standard-library C++ packages, thus

enabling a trivial setup of this project. Each SG fulfils an interface containing essential methods for planners such as sampling an initial state, copying states, obtaining the set of legal actions for a state, applying an action to a state, iterating over all successors of a state-action pair, obtaining the sample probability for any state-action pair successor (except for Wildlife Preserve and Push Your Luck because that calculation would be infeasible in these domains), hashing a state, or test any two states for equality. A high-level UML diagram of these interfaces is shown in Fig. 1, which shows that our implementation separates between Gamestate objects and Model objects, which are used to apply operations to the Gamestate objects.

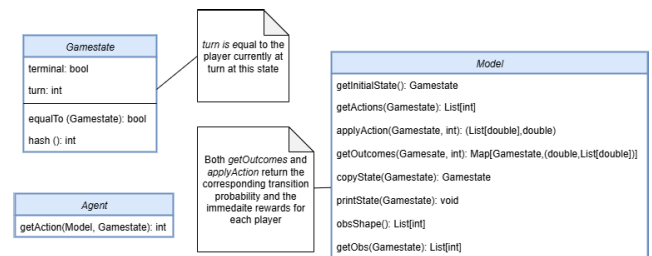


FIGURE 1. A high-level UML overview of the interfaces provided by BenchmarkGames.

In particular, most of the 22 stochastic MDPs thus far have only been accessible through an RDDDL interpreter in Python [11], which is several thousand times slower than our C++ interface, as will be shown in Section IV. In particular, this speed-up made the experiments of previous work [2] feasible in the first place, which tested several thousand parameter combinations of an MCTS modification.

A. REFERENCE AGENTS

Furthermore, our implementation also provides C++ reference agents from the literature, including a random, MCTS, Sparse Sampling, Progressive Abstraction Refinement for Sparse Sampling [9], Abstractions of State-Action Pairs [4], and an FF-Hindsight [12] agent.

B. REINFORCEMENT LEARNING SUPPORT

Though the environments were not intended to be used for reinforcement learning, we additionally implemented an observation and action encoding method for most environments. For further accessibility, we also provide a Python interface for these implementations that satisfies OpenAI's gym interface [13] that additionally implements the above-mentioned planning methods.

III. A CATALOGUE OF MDPs AND TWO-PLAYER SGs

Next, to address the second hurdle, which is to find suitable environments with the right characteristics to run experiments on, this article provides a catalogue containing a description of each of the implemented environments with a focus on their characteristics. This catalogue is provided in Section B (for MDPs) and in Section C (for two-player

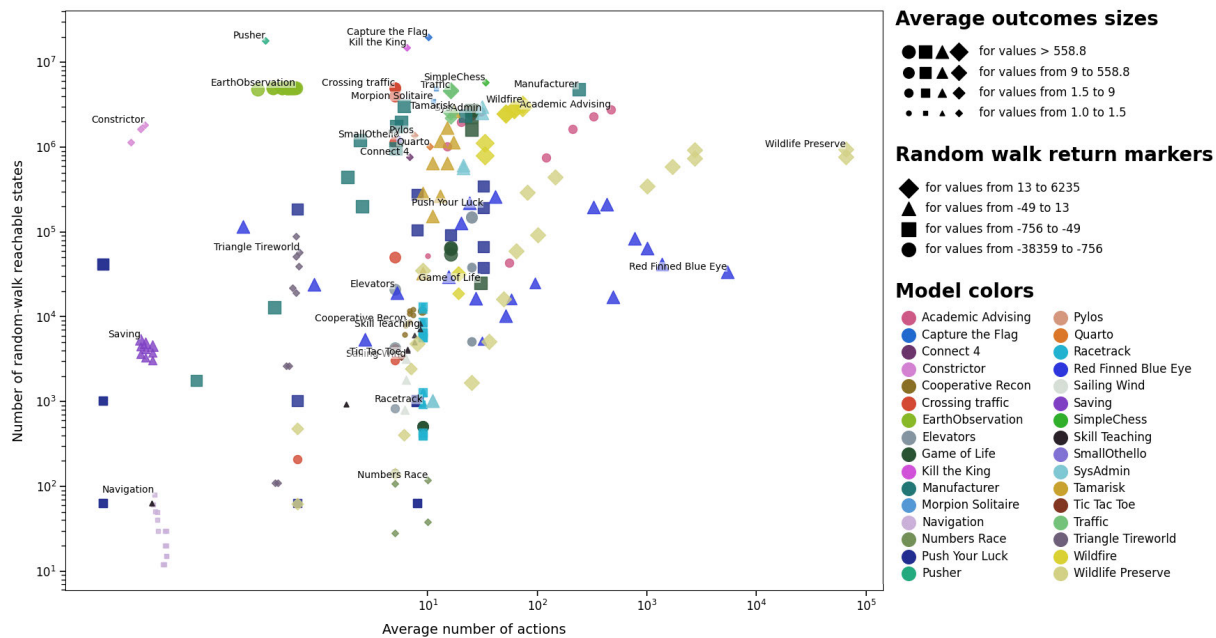


FIGURE 2. A scatter plot of the properties of all of the implemented stochastic MDPs and two-player SGs along with their instances. For each problem model, the instance at the end of each of the following data tables has been labelled. The data has been generated with the software accompanying this article.

SGs) in the Appendix. Furthermore, the catalogue includes a statistical analysis of each environment to approximate model properties that are mostly related to the model's complexity, like the average number of state-action pair outcomes or the average number of legal actions. These properties correspond to the metrics category of the game concept taxonomy proposed by Piette et al. [14]. A visual overview of the various models in dependence of a subset of their characteristics is given in Fig. 2. We note that the Ludii framework [15] also comes with a tool for finding suitable environments called the *Concept Search Database*, however, this is of little use to the probabilistic planning community as it contains next to none of the literature-relevant domains. Hence, within the space of probabilistic planning, this such a tool for task selection did not yet exist.

IV. COMPARISON TO OTHER FRAMEWORKS

Next, we will be comparing our stochastic MDP implementations, which we will be referring to as *BenchmarkGames*, against the Griddly [16], PyRDDL [11], Ludii [15], MDP-Engine [17], OpenSpiel [18], and PGX [19] frameworks, which all provide access to a number of MDPs (and more generally SGs). We did not include the frameworks MiniGrid [20], General Video Game AI [21], or DeepMind's Lab2D [22] as Griddly was already compared to these [16], showing that Griddly was clearly superior. Tab. 1 lists the different features of these frameworks. For each framework, this table counts the total number of implemented problem models, which may include continuous-action or imperfect-information models. In addition, it also counts the number of

those models that could be formalized as non-deterministic MDPs. Note that we also counted imperfect-information models towards this number, as one could just ignore the hidden state for planning. The table has been created with the state of these codebases on the 16th of September 2025.

The environments, *BenchmarkGames* is compared against, can be split into two categories. Those that are interpreter-based and those that feature manually implemented environments. The former category consists of Ludii, PyRDDL, and Griddly, while the latter category is made up of *BenchmarkGames*, MDP-Engine, OpenSpiel, and PGX.

Though the interpreter-based framework PyRDDL features more stochastic MDPs than *BenchmarkGames*, the interpreter-based approach comes at the cost of runtime overheads. Table 2 compares the average forward model call time without observation calculation in milliseconds on an arbitrarily selected subset of games to illustrate the clear runtime advantage *BenchmarkGames* has over PyRDDL, Griddly, and Ludii. More precisely, the data was obtained by calling the step method on random actions on states obtained by running a random policy. The measurements were taken on a machine with the processor AMD Ryzen 7 6800H with Radeon Graphics, 3201 MHz, with 8 cores. For Navigation, Game of Life, and Academic Advising, we used the 10th IPPC problem instance, and for Sailing Wind, a grid of side length 15. For Kill the King and Pusher, we used the instance *KillTheKing.yaml* with the first map of *KtKMaps_kwah.yaml* available at the repository <https://github.com/egg-west/Stratega/tree/master> which was

TABLE 1. Comparison of the features of different SG simulator interfaces with our framework (BenchmarkGames).

Property	BenchmarkGames (ours)	Ludii [15]	PyRDDL [11]	Griddly [16]	MDP-Engine [17]	OpenSpiel [18]	PGX [19]
Implemented environment types	Stochastic MDPs and two-player board- and strategy-games	Stochastic/Deterministic, Single/Multiplayer, Perfect/Imperfect Information games	Stochastic MDPs	MDPs and multi-agent games with a focus on strategy-games	Stochastic MDPs	Stochastic/Deterministic, Single/Multiplayer, Perfect/Imperfect Information games	Board games and Atari
# Problem models	33	1621	66	32	7	88	20
# Non-deterministic MDPs	22	9	33	5	7	6	6
Languages of supported interfaces	Python, C++	Java	Python	Python	C++	Python, C++	Python
Has observations?	Yes	No	Yes	Yes	No	Yes	Yes
Can copy states?	Yes	Yes	Yes (but not part of the interface)	Yes	Yes	Yes	No
Can iterate action outcomes?	Yes	Yes	No	No	Yes	Yes	No
Has transition probabilities?	Yes	No	No	No	Yes	Yes	No
Has equality-/hash-operator?	Yes	Yes	No	No	Only equality	Implicitly in the form of serialization	No
Visual output	Visualization as strings	2-D rendering	None	2-D rendering	None	Visualization as strings	SVG

TABLE 2. Comparison of the forward model call execution time in milliseconds of BenchmarkGames (our implementation) versus other frameworks. All measurements have been rounded to three decimal points, except the PGX and C++ BenchmarkGames interface times, which are rounded to five decimal points. The PGX columns shows the measurements for batched forward calls (left) using the batch size 4048 and the measurements for non-batched forward calls (right).

Environment	BenchmarkGames (ours)		Ludii (Java)	PyRDDL (Python)	Griddly (Python)	MDP-Engine (C++)	OpenSpiel (Python)	PGX
	Python Interface	C++ Interface						
Navigation	0.006 ms	0.0002 ms	N/A	0.501 ms	N/A	N/A	N/A	N/A
Game of Life	0.003 ms	0.0006 ms	N/A	0.493 ms	N/A	N/A	N/A	N/A
Academic Advising	0.006 ms	0.0002 ms	N/A	0.588 ms	N/A	N/A	N/A	N/A
Kill the King	0.005 ms	0.0002 ms	N/A	N/A	0.013 ms	N/A	N/A	N/A
Pusher	0.006 ms	0.0002 ms	N/A	N/A	0.013 ms	N/A	N/A	N/A
Connect4	0.005 ms	0.0002 ms	0.014 ms	N/A	N/A	N/A	0.0002 ms	0.0002 ms / 0.089 ms
Morpion Solitaire	0.006 ms	0.001 ms	14.753 ms	N/A	N/A	N/A	N/A	N/A
Tic Tac Toe	0.004 ms	0.0001 ms	N/A	N/A	N/A	N/A	0.0001 ms	0.0001 ms / 0.031 ms
Sailing Wind	0.006 ms	0.0002 ms	N/A	N/A	N/A	0.001 ms	N/A	N/A

used by Xu et al. for their Elastic Monte Carlo Tree Search paper [10]. To put our C++ interface times into perspective, with an average execution time of 0.0002 ms, which is achieved by multiple models, one could run roughly 50000 random walk episodes with a horizon of 100 per second. BenchmarkGame's C++ interface is in all considered SGs at least 60 times faster than any of the interpreter-based frameworks, and the speedup can go as high as $\approx 10^5$ in the case of Morpion Solitaire.

On the contrary, while libraries like MDP-Engine, PGX, or OpenSpiel with manually implemented games may not suffer from the same runtime issues, they are limited in the number of implemented environments. Even though OpenSpiel has implemented 88 problem models, only 6 of those are non-deterministic MDPs, one of the main subjects of interest for probabilistic planning research. Hence, BenchmarkGames, containing almost four times as many stochastic MDPs, closes this gap.

V. CONCLUSION AND FUTURE WORK

In conclusion, if one aims to conduct probabilistic planning research on stochastic MDPs and one does not require fancy visualizations, we believe that our implementation

TABLE 3. Academic advising parameters.

Parameter	Description
File	Path to the model instance to be used.
Idle	Whether the agent is allowed to idle, i.e., choose no courses.
Dense rewards	Whether the agent receives dense rewards.

BenchmarkGames is currently the best choice that drastically accelerates research. In comparison to the MDP-Engine, OpenSpiel, or PGX, BenchmarkGames encompasses more than three to four times as many stochastic MDPs whilst offering the same or a larger set of features. Though there exists a interpreter-based approach with access to more stochastic MDPs than BenchmarkGames, PyRDDL's forward model is orders of magnitude slower to query than BenchmarkGames. Future work will be to expand the collection of environments.

APPENDIX A

DEFINITION OF MARKOV DECISION PROCESSES (MDP) AND STOCHASTIC GAMES (SG)

All environments that will be presented in the following can be formalized as finite SGs. We use the following definition

TABLE 4. Academic advising model statistics for non-dense rewards and idling not being allowed.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-236.51	10.00	1.47	42.93	52664
(2_IPPC)	N/A	-288.53	55.00	1.52	50.00	42931
(3_IPPC)	N/A	-285.81	15.00	1.53	49.68	1025100
(4_IPPC)	N/A	-394.01	120.00	1.78	50.00	747772
(5_IPPC)	N/A	-329.61	20.00	1.63	50.00	1966880
(6_IPPC)	N/A	-408.54	210.00	2.03	49.98	1630953
(7_IPPC)	N/A	-327.57	25.00	1.70	50.00	2347533
(8_IPPC)	N/A	-409.81	325.00	2.20	50.00	2298882
(9_IPPC)	N/A	-325.49	30.00	1.75	50.00	2613184
(10_IPPC)	N/A	-409.04	465.00	2.37	50.00	2775842

for SGs. We use $\Delta(X)$ to denote the probability simplex of a finite, non-empty set X .

Definition: A finite SG is a 9-tuple $(n, S, F, \mu_0, h, A, T, \mathbb{P}, R)$ where $n \in \mathbb{N}$ is the number of players, $S \neq \emptyset$ is the finite set of states, $F \subsetneq S$ is the set of terminal states, $\mu_0 \in \Delta(S \setminus F)$ is the probability distribution for the initial state, and $h \in \mathbb{N}$ is the horizon length. Furthermore, $A: (S \setminus F) \mapsto \{X \subseteq \mathbb{Z} \mid \emptyset \neq X, |X| < \infty\}$ maps each state to its legal actions which are coded as integers. We denote the set of legal state-action pairs as $P = \{(s, a) \mid s \in (S \setminus F), a \in A(s)\}$. Next, $T: S \mapsto \{1, \dots, n\}$ maps each state to the player whose turn it is, $\mathbb{P}: P \mapsto \Delta(S)$ is the stochastic transition function where we use $\mathbb{P}(s' \mid s, a)$ to denote the probability of transitioning from $s \in (S \setminus F)$ to $s' \in S$ after taking action $a \in A(s)$ in s . Lastly, $R: P \mapsto \mathbb{R}^n$ is the joint reward function with R_i being the reward function for player $1 \leq i \leq n$.

Starting with $s_0 \sim \mu_0$, a SG progresses from state s_t to s_{t+1} according to $\mathbb{P}$ given the action $a_t \in A(s_t)$ decided by the player $T(s_t)$. Depending on the concrete problem setting, the game ends either after $h \in \mathbb{N}$ steps or earlier if $s_{t+1} \in F$. Each player's goal is to maximize one's own, possibly discounted, sum of obtained rewards.

An SG is also an MDP if $n = 1$. An SG (and analogously an MDP) is called non-deterministic if there exists an $((s, a), s') \in P \times S$ such that $\mathbb{P}(s' \mid s, a) \notin \{0, 1\}$. All SGs that we consider in this paper are either MDPs or $n = 2$ i.e., two-player games.

APPENDIX B

CATALOGUE OF MARKOV DECISION PROCESSES (MDP)

In this section, we give a description for each MDP that we implemented. For each environment, we first give a high-level overview of the environment, then we describe the action space, the reward function, sources of stochasticity, and terminal states in more detail. Additionally, we provide a table of parameters to customize the environments. Furthermore, we ran a random agent on various model parameter combinations for 100000 games using a horizon of 50 steps (and 200 steps for Morpion Solitaire) and we list the average return V_{rdm} , the average number of legal actions, the total number of encountered states, the average episode length, and the average number of state-action-pair successors for each model instance. *Note:*

TABLE 5. Cooperative recon parameters.

Parameter	Description
File	Path to the model instance to be used.

We performed this random walk analysis as calculating these values exactly is intractable for most environments. The software accompanying this article, available at https://github.com/codebro634/Benchmark_Games.git, has an extensive README, which can be used to reproduce these statistics.

For those environments where we could solve the environment exactly with value iteration, we also denote the optimal initial state(s) value V^* as well as the ratio of local state pairs and action pairs that are value-equivalent. These ratios were determined by randomly sampling 100000 states. We then applied $i \in \{1, 2, 3\}$ random actions to each state and a copy of each state. We then counted how many times out of these 100000 states, the resulting states after applying i actions, had the same value or the same Q-value for the i -th action. We will later denote these ratios by $V_{\text{abs}}(i)$ and $Q_{\text{abs}}(i)$. Note that we calculate these statistics with respect to the finite horizon version of the respective environment, meaning that some abstractions only exist due to the episode's cutoff at the end of the horizon.

The MDPs that we implemented are either problems from the International Probabilistic Planning Competition [23] (IPPC), well-known single-player board games, and/or used/proposed by different researchers. For each environment, we state where it has been used. In the case of the IPPC problems, for consistency with existing work, we reimplemented the problem's RDDDL descriptions from the RDDDL-repository [11], which we refer to as PYRDDDL. We always clearly state any changes made to the PYRDDDL versions, which could be due to known bugs of the RDDDL version or to allow for more customizability by parametrizing the environment.

A. ACADEMIC ADVISING

Description: The agent represents a student aiming to pass a set of academic courses. Formally, the state is defined as

TABLE 6. Cooperative recon model statistics.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-3.56	8.81	1.56	50.00	11795
(2_IPPC)	N/A	-0.87	8.83	1.36	50.00	11419
(3_IPPC)	N/A	-1.00	7.29	1.43	50.00	12085
(4_IPPC)	N/A	-2.32	7.32	1.35	50.00	10476
(5_IPPC)	N/A	-1.24	6.76	1.20	50.00	8596
(6_IPPC)	N/A	-2.58	6.83	1.18	50.00	11660
(7_IPPC)	N/A	-2.46	6.77	1.22	50.00	10959
(8_IPPC)	N/A	0.26	6.21	1.10	50.00	7827
(9_IPPC)	N/A	-0.67	6.13	1.18	50.00	6216
(10_IPPC)	N/A	-1.32	6.37	1.10	50.00	8305

an element of $\{P, NP, NT\}^{N_{\text{courses}}}$, indicating for each course whether it has been passed (P), not passed (NP), or not yet taken (NT). At each step, the agent selects a subset of courses to take concurrently. The success of these courses depends on the completion status of their prerequisites. As long as not all required (i.e., mandatory) courses are completed, the agent incurs a fixed penalty at each time step.

Action space: Depending on the concrete instance, the agent may choose between 1 and 2 courses per step to take. If the idle action parameter is set, then the agent is allowed to take no courses.

Rewards: The reward $r = r_t + r_a$ function can be decoupled into an action-independent component r_t and an action dependent component r_a . The former depends on whether the dense reward parameter is set. In the case of dense rewards, the reward is independent of the chosen action and equal to

$$r_t = 5 \left(\frac{N_{\text{passed}}}{N_{\text{courses}}} - \frac{N_{\text{missing_mand}}}{N_{\text{mand}}} \right) \quad (1)$$

where N_{passed} is the number of passed courses, N_{courses} is the number of courses, $N_{\text{missing_mand}}$ is the number of missing mandatory courses, and N_{mand} is the number of mandatory courses. In case dense rewards are not set, r_t is equal to 0 if all mandatory courses have been passed and otherwise -5 . The action-dependent reward r_a is equal to

$$-1N_{\text{taken}} - 2N_{\text{redo}} \quad (2)$$

where N_{taken} is the number of courses taken at the current step (i.e. 0,1, or 2) that have not been taken previously and N_{redo} is the number of courses taken at the current step that have already been taken at some previous step.

Stochasticity: For each course to be taken that has not yet been passed, the agent may pass the course with probability p or fail with probability $1 - p$, leaving the state of that course unchanged. The courses form a directed dependency graph. The pass probability p is always less than 1 but increases linearly with the number of passed prerequisites. The slope is approximately inversely proportional to the number of prerequisites.

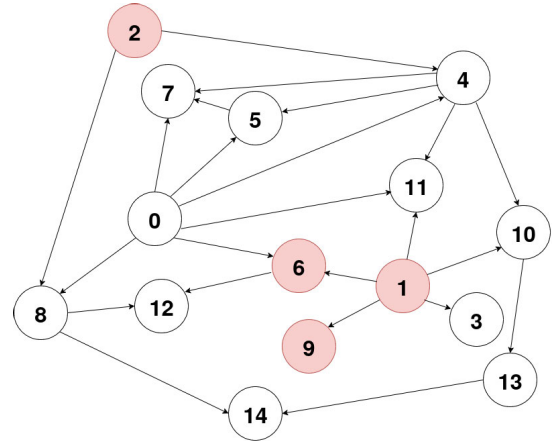
Terminal states: The episode ends prematurely if the goal state is reached.

Initial states: Initially, no courses are taken.

TABLE 7. Crossing traffic parameters.

Parameter	Description
Idle	Whether the idle action is allowed.
p_{spawn}	The probability with which obstacles spawn.
$n \in \mathbb{N}$	The width of the grid.
$m \in \mathbb{N}$	The height of the grid.

Notes: The Academic Advising problem was originally proposed by Guerin et al. [24] and was slightly adapted for the IPPC 2014 [25]. Unlike in the PYRDDL, we allow for an early episode termination if all courses are passed. In the PYRDDL version, dense rewards are not set, and idling is allowed. Fig. 3 visualizes the course dependency graph of the Academic Advising instance 2_Anand. In total, this project comes with 10 pre-implemented instances.

**FIGURE 3. The requirements graph of the Academic Advising instance 2_Anand. The courses marked in red are the target classes.**

B. COOPERATIVE RECON

Description: This domain models a robot having to prove the existence of life on a foreign planet. The robot is modeled as moving on a 2-dimensional grid which contains a number of objects of interest and a base. If the agent is at an object of interest, it can survey the object for the existence of water and life. The probability of a positive result of the latter

TABLE 8. Crossing Traffic model statistics. The format is (n, m, p_{spawn}) .

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(4,3,0.5)	-5.00	-43.27	4.00	1.86	50.00	208
(4,4,0.4)	-6.78	-48.66	5.00	3.92	50.00	4267
(4,4,0.6)	-10.5	-49.58	5.00	3.97	50.00	4197
(4,4,0.8)	-25.17	-49.93	5.00	4.00	50.00	3046
(6,6,0.4)	N/A	-49.95	5.00	15.98	50.00	4071017
(6,6,0.6)	N/A	-50.00	5.00	16.00	50.00	3984834
(6,6,0.8)	N/A	-50.00	5.00	16.00	50.00	1135465
(8,8,0.4)	N/A	-50.00	5.00	64.00	50.00	4999903
(8,8,0.6)	N/A	-50.00	5.00	64.00	50.00	5000000
(8,8,0.8)	N/A	-50.00	5.00	64.00	50.00	4953551

TABLE 9. Crossing traffic statistics.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(4,3,0.5)	0.82	0.89	0.84	0.88	0.85	0.88
(4,4,0.4)	0.86	0.87	0.87	0.86	0.86	0.86
(4,4,0.6)	0.90	0.90	0.90	0.89	0.90	0.89
(4,4,0.8)	0.91	0.91	0.92	0.91	0.92	0.91

TABLE 10. Earth Observation parameters.

Parameter	Description
File	Path to the model instance to be used.

is dependent on whether water has been detected. If life has been detected, the agent may photograph the object of interest, which is the only way to gain a reward. Each detector may break on usage, making it either unusable or decreasing its chance of working. The detectors can be repaired at the base. Furthermore, the likelihood of detectors breaking is increased within the proximity of hazards that are spread around the grid.

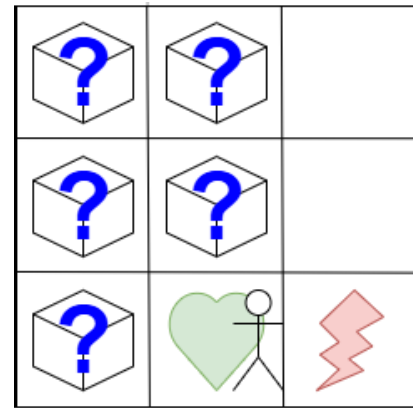
Action space: At each step, the agent can move to any of the four adjacent tiles. Though actions that would move the agent outside the grid are allowed, they have no effect. If the agent is at the base-tile, three repair actions become available, one for each tool (e.g., camera, life-detector, water-detector). If the agent is at an object of interest, three actions that correspond to using any of the tools become available.

Rewards: The reward is always 0 except when the agent photographs an object of interest where life has been detected. In that case, a positive reward is received that depends on whether the camera is damaged. The concrete reward values are instance dependent.

Stochasticity: Each tool that is not yet damaged has a tool-specific probability to break at each step. Additionally, if the agent uses detect life on an object of interest where water has previously been detected, there is a 0.8 chance to detect life if the life detector is not damaged and only a 0.4 chance if the detector is damaged.

Terminal states: There are no terminal states.

Initial states: Initially, all tools are undamaged. The initial grid position is instance-dependent.

**FIGURE 4.** A visualization of the Cooperative Recon instance 3_IPPC. Question mark cells indicate objects of interest, the lightning represents a hazard, the green heart the base for repairs, and the stick figure is the initial position of the agent.

Notes: Cooperative Recon was a domain for the IPPC 2018. Fig. 4 shows the Cooperate Recon instance 3_IPPC. In total, this project comes with 10 pre-implemented instances.

C. CROSSING TRAFFIC

Description: Crossing Traffic is a grid-based navigation task in which the agent must traverse multiple lanes of moving traffic. Obstacles move horizontally from left to right along the x-axis and spawn randomly at the right edge of the grid, excluding the first and last rows. If the agent collides with an obstacle, it becomes immobilized, making it impossible to complete the episode. The agent starts at the top-left corner of the grid and aims to reach the bottom-left cell in the fewest possible steps while avoiding collisions.

Action space: At each step, the agent may move to any of the 4 neighboring tiles, if they would not move the agent outside the grid. Additionally, the agent may idle if the idle parameter has been set.

Rewards: If the bottom left goal cell has been reached, the agent receives a reward of 0 until the rest of the episode; otherwise, the agent is punished with a reward of -1 .

Stochasticity: Except for the first lane (where the agent spawns) and the last lane (which includes the goal cell), a new obstacle may randomly spawn with a fixed probability p_{spawn} .

TABLE 11. Earth Observation model statistics.

Instance	V^*	V_{randm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-75.65	3.59	N/A	50.00	4838020
(2_IPPC)	N/A	-1017.63	3.75	N/A	50.00	4961288
(3_IPPC)	N/A	-1088.06	3.84	N/A	50.00	4998878
(4_IPPC)	N/A	-1672.99	3.89	N/A	50.00	5000000
(5_IPPC)	N/A	-796.46	3.92	N/A	50.00	5000000
(6_IPPC)	N/A	-588.05	3.75	N/A	50.00	4999999
(7_IPPC)	N/A	-170.73	3.84	N/A	50.00	5000000
(8_IPPC)	N/A	-649.96	3.89	N/A	50.00	5000000
(9_IPPC)	N/A	-1377.40	3.92	N/A	50.00	5000000
(10_IPPC)	N/A	-2651.28	3.95	N/A	50.00	5000000
(11_IPPC)	N/A	-1191.34	3.84	N/A	50.00	5000000
(12_IPPC)	N/A	-1908.69	3.89	N/A	50.00	5000000
(13_IPPC)	N/A	-1585.65	3.92	N/A	50.00	5000000
(14_IPPC)	N/A	-6064.68	3.95	N/A	50.00	5000000
(15_IPPC)	N/A	-1520.50	3.97	N/A	50.00	5000000
(16_IPPC)	N/A	-644.32	3.89	N/A	50.00	5000000
(17_IPPC)	N/A	-1721.50	3.92	N/A	50.00	5000000
(18_IPPC)	N/A	-2281.24	3.95	N/A	50.00	5000000
(19_IPPC)	N/A	-2949.55	3.97	N/A	50.00	5000000
(20_IPPC)	N/A	-6680.03	3.98	N/A	50.00	5000000

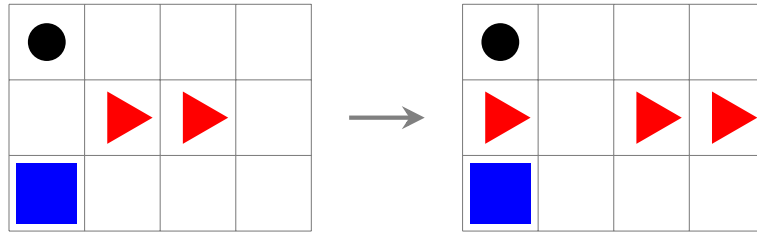


FIGURE 5. A visualization of a sample state transition for a Crossing Traffic instance on a 3×4 grid, assuming that the player (black dot) chose to idle. Red triangles indicate obstacles, and the blue square is the goal position.

Terminal states: There are no terminal states.

Initial states: The agent is initially placed at position (0, 0) with lanes being populated randomly.

Notes: Crossing traffic was a test problem for the IPPC 2014. The PYRDDL version allows idling. Fig. 5 shows a sample state transition of a 3×4 Crossing Traffic grid. What is challenging about Crossing Traffic is that a crash does not directly lead to a terminal state but simply renders the agent unable to receive any reward down the line.

D. EARTHOBSERVATION

Description: EarthObservation simulates a satellite orbiting the Earth. Formally, each state consists of the current weather conditions across all regions (modeled as vertices in a directed graph), the vertex where the agent is located (indicating the region the satellite's camera is focused on), and a set of marked target regions of interest. Weather conditions evolve stochastically at each time step, independent of the agent's actions, which control the satellite's movement. The agent's goal is to capture clear images of the target regions. Once a target is successfully photographed, it is removed from the active target list.

Action space: At each step, the agent has access to three or four actions which correspond to slewing north-east, slewing south-east, slewing east, and slewing east plus taking an

image. The last action is only available if the current state is a region of interest. Both the last and second-to-last action transition to the same vertex or region the satellite's camera is aimed at.

Rewards: The reward is equal to $n_{\text{targets}} - r_a$ where n_{targets} is the number of regions of interest that have not yet been successfully photographed, $r_a = -2$ if the action was slewing east and taking an image, $r_a = 0$ if slewing east, and $r_a = -1$ for all other actions.

Stochasticity: Each cell can have one of three weather conditions, which change randomly and independently from one state to the next. If the agent's camera is aimed at a region of interest and the photograph action has been taken, the photograph may randomly fail with a probability depending on the region's current weather condition.

Terminal states: There are no terminal states.

Initial states: The initial visibility and the satellite's positions are instance-dependent.

Notes: EarthObservation was proposed by Hertle et al. [26] and used as a problem for the IPPC 2018. In total, this project comes with 20 pre-implemented instances. The challenge about EarthObservation is for the agent to deal with the enormously large stochastic branching factor whose outcome branches are effectively equivalent.

TABLE 12. Elevators parameters.

Parameter	Description
File	Path to the model instance to be used.

E. ELEVATORS

Description: In Elevators, the agent simultaneously controls a number of elevators with the aim of picking up and carrying passengers that randomly arrive at different floors as quickly as possible to their desired floor, which can either be the highest or lowest floor. Per floor, there is at most one person waiting for an elevator going up, and at most one waiting for an elevator going down. Each elevator can carry an arbitrary number of people who want to go up or down, even when they have conflicting destinations. Once an elevator reaches the top or bottom floor, all passengers whose destination is this was are disembarked.

Action space: For each of the instance's e elevators, the agent has to decide whether to idle, to close the door, to move in the current direction, or to open the door while signifying that the elevator will travel up/down. Hence, the action space is of size 5^e .

Rewards: The reward r is action independent and given by

$$r = -n_{\text{waiting}} - 0.75 \cdot n_{\text{right}} - 3 \cdot n_{\text{wrong}} \quad (3)$$

where n_{waiting} denotes the number of persons at all floors that are waiting for an elevator, n_{right} denotes the number of elevators that carry at least one person in the right direction and n_{wrong} the number of elevators that carry at least person in the wrong direction.

Stochasticity: Passengers arrive randomly at each floor f with an instance and floor specific probability p_f that is 0 for the bottom and top floor. This probability is the same, independent of the arriving passenger's destination. It is possible for two passengers (one for each direction) to arrive in the same step at the same floor.

Terminal states: There are no terminal states.

Initial states: Initially, there are no waiting passengers, and all elevators start at the bottom floor with the direction up.

Notes: Elevators has been a problem for the IPPC 2014. The PYRDDL version allows idling. In total, this project comes with 10 pre-implemented instances.

F. GAME OF LIFE

Description: In Game of Life, the agent is tasked with keeping as many cells alive as possible. States are elements in $\{0, 1\}^{n \times m}$ with fixed n, m and correspond to a grid of cells, each of which is either dead or alive. The grid evolves according to the standard Game of Life rules by John Conway [27] with the exception that the agent is allowed to specify one cell that will live in the next step and that each cell may randomly and independently change its state.

Action space: The agent is allowed to choose any cell that will live with a high probability in the next step. If the save-

only parameter is specified, the agent may only choose a cell that is currently alive. And if the revive-only parameter is specified, the agent may only select a cell that is currently dead.

Rewards: The reward is only state-dependent and equal to the number of living cells.

Stochasticity: After the deterministic rules have been applied (i.e., automaton rules + action override), each cell (i, j) has a chance $p_{i,j}$ to swap its state (i.e., alive to dead or dead to alive). Hence, the number of outcomes is equal to the state space size, which is equal to $2^{m \cdot n}$.

Terminal states: There are no terminal states.

Initial states: The initial configuration of living cells is instance-dependent.

Notes: This environment is derived from the cellular automaton Game of Life by John Conway [27] and used as a test problem for the IPPC 2011 where the full action space was used. Anand et al. [4] introduced the revive-only variant. Fig. 6 depicts the initial state of the Game of Life instance 3_Anand. In total, this project comes with 10 pre-implemented instances independent of the chosen action space.

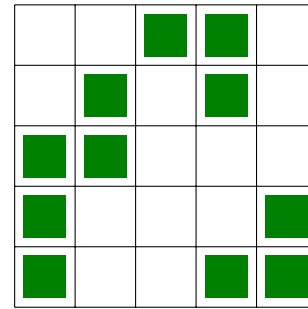


FIGURE 6. A visualization of the initial state used for the 5×5 Game of Life instance 3_Anand. Green cells indicate living cells.

G. MANUFACTURER

Description: In this domain, the agent manages a manufacturing company. The agent's ultimate goal is to sell goods to customers. However, to sell a good, the agent has to first produce the good, which may require building factories and acquiring the necessary goods required for production. Additional difficulty comes from the fact that the goods' price levels vary stochastically.

Action space: The agent can spend a number of action points equal to its number of factors on the following actions, which all cost one action point each. For each good, the agent may choose one of six actions: purchase an ingredient good, produce the good, or sell the good. There are also composite actions that chain these, such as purchase-produce-sell. The agent can also execute at most one general-management action, which is to build a factory, hire a production manager for a specific good (only if there is already a factory for that good), hire a lobbyist, or a marketing manager. However, hiring is only possible if such a manager has applied.

TABLE 13. Elevators model statistics.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	-56.53	-111.22	5.00	1.72	50.00	144
(2_IPPC)	-21.36	-107.20	25.00	2.38	50.00	5102
(3_IPPC)	-65.30	-150.05	25.00	1.82	50.00	5144
(4_IPPC)	-70.55	-155.59	5.00	4.75	50.00	832
(5_IPPC)	N/A	-192.23	25.00	4.99	50.00	38321
(6_IPPC)	N/A	-213.75	25.00	4.26	50.00	37850
(7_IPPC)	-104.86	-207.32	5.00	13.63	50.00	4344
(8_IPPC)	N/A	-241.00	25.00	13.44	50.00	151344
(9_IPPC)	N/A	-267.68	25.00	11.39	50.00	148387
(10_IPPC)	-88.20	-188.59	5.00	63.13	50.00	20937

TABLE 14. Elevators abstraction statistics.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC)	0.23	0.47	0.22	0.38	0.21	0.34
(2_IPPC)	0.21	0.34	0.18	0.24	0.16	0.20
(3_IPPC)	0.18	0.33	0.16	0.22	0.15	0.19
(4_IPPC)	0.20	0.44	0.17	0.33	0.15	0.28
(7_IPPC)	0.18	0.43	0.15	0.31	0.12	0.25
(10_IPPC)	0.18	0.43	0.15	0.31	0.12	0.24

The following additional constraints have to be met: Goods can only be produced if a factory for that good has been built, all prerequisites goods have to be in stock before a good can be produced, only goods in stock can be sold, purchase can only be chained with produce and sell if a production manager has been hired. Furthermore, one can only purchase or produce goods that are required for at least one other good, and factories can only be built if a factory for that good does not yet exist and is not already in construction. Lastly, some goods are marked as prime goods and cannot be produced by a factory.

Rewards: The immediate reward r has the form

$$\left(\sum_{g \in G_s} \text{pr}(g) \cdot w(l(g)) \cdot (1 + b_{\text{mm}}) \right) + \left(\sum_{g \in G_b} \text{pr}(g) \cdot w(l(g)) \cdot (1 - b_{\text{lb}}) \right) + \left(\sum_{g \in G_f} \text{build}(g) \right) + r_{\text{salary}} \quad (4)$$

where G_s is the set of all goods that have been sold, G_b is the set of all bought goods, G_f is the set of all factories under construction, $\text{pr}(g)$ is the instance-specific good price, $l(g)$ is the current price level of g , $w(l)$ is an instance-specific factor associated with a price level, b_{mm} is an instance-specific bonus a marketing manager yields, and b_{lb} is the analogue for a lobbyist, $\text{build}(g)$ denotes the factory construction cost for that good, and lastly

$$r_{\text{salary}} = -r_{\text{pm}} \cdot |G_{\text{pm}}| - r_{\text{mm}} - r_{\text{lb}} \quad (5)$$

with r_{pm} denoting the instance-specific salary of production managers and r_{mm} and r_{lb} denote the salary of the marketing manager or lobbyist, and G_{pm} is the set of all goods for which a production manager has been hired. All these values are only non-zero if the corresponding staff has been hired.

Stochasticity: There are three sources of stochasticity. Firstly, the price level $l(g)$ randomly fluctuates for each good g according to an instance-specific transition probability matrix that is also dependent on the price trend for $t(g)$ of g . There are three price trends: Up, Down, and Stable, which also randomly fluctuate according to an instance-specific transition probability matrix, with the transition from Up to Down or Down to Up having zero probability. Additional stochasticity comes from all three manager types applying or retracting their application randomly with instance-specific probabilities at each step.

Lastly, the construction of factories, which takes a minimum of 4 steps, can randomly be delayed.

Terminal states: There are no terminal states.

Initial states: Initially, there are no goods in stock, all price levels are at level 4 (there are 7 different price levels), and no managers have applied. The initial price trends and starting factories are instance-specific.

Notes: Manufacturer has been a problem for the IPPC 2018. Fig. 7 visualizes the production dependency graph of

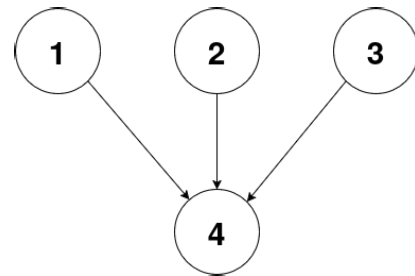


FIGURE 7. Good production dependency graph of the Manufacturer instance 3_IPPC. An arrow from node i to node j indicates that product j requires product i for production.

TABLE 15. Game of Life parameters.

Parameter	Description
File	Path to the model instance to be used.
Actions	Specifies the actions space. Possible values are all, save-only, revive-only.

TABLE 16. Game of Life model statistics when all actions are allowed.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC,all)	288.03	126.72	9.00	512.00	50.00	512
(2_IPPC,all)	217.80	142.04	9.00	512.00	50.00	512
(3_IPPC,all)	234.41	179.14	9.00	512.00	50.00	512
(4_IPPC,all)	N/A	231.96	16.00	N/A	50.00	54116
(5_IPPC,all)	N/A	297.68	16.00	N/A	50.00	63311
(6_IPPC,all)	N/A	331.28	16.00	N/A	50.00	65207
(7_IPPC,all)	N/A	361.85	25.00	N/A	50.00	1595607
(8_IPPC,all)	N/A	412.29	25.00	N/A	50.00	2281575
(9_IPPC,all)	N/A	443.44	25.00	N/A	50.00	2746666
(10_IPPC,all)	N/A	713.52	30.00	N/A	50.00	25007

TABLE 17. Game of Life abstraction statistics. Each instance has the format (File,Actions).

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC,revive_only)	0.28	0.06	0.25	0.03	0.24	0.02
(1_IPPC,save_only)	0.02	0.66	0.02	0.30	0.02	0.18
(1_IPPC,all)	0.40	0.09	0.37	0.03	0.35	0.02
(2_IPPC,save_only)	0.03	0.43	0.02	0.07	0.02	0.03
(2_IPPC,revive_only)	0.15	0.06	0.12	0.01	0.10	0.01
(2_IPPC,all)	0.21	0.10	0.17	0.02	0.15	0.01
(3_IPPC,save_only)	0.03	0.39	0.02	0.04	0.01	0.01
(3_IPPC,revive_only)	0.06	0.07	0.04	0.01	0.03	0.01
(3_IPPC,all)	0.10	0.13	0.07	0.01	0.06	0.01

TABLE 18. Manufacturer parameters.

Parameter	Description
File	Path to the model instance to be used.

Manufacturer instance 3_IPPC. In total, this project comes with 11 pre-implemented instances.

H. MORPION SOLITAIRE

Description: Morpion Solitaire is played on a 2-D grid with an initial configuration of marked cells as visualized in Fig. 8. The goal is to draw as many lines as possible that connect exactly five points satisfying certain conditions.

Action space: The agent may at each step draw a horizontal, vertical, or diagonal line that connects exactly 5 points. Four of these points must have been previously marked. The fifth point is marked after the action has been applied. Parallel lines may only intersect at their start or end points (joint version) or not intersect at all (disjoint version). If the decoupled action space option is set, then each original action is split into two steps. In the first step, a cell has to be chosen which must be one of the available lines' endpoints. In the second step, the second endpoint is chosen.

Rewards: At each step, the agent receives a reward of 1. Hence, the objective is to maximize the number of moves before the episode terminates.

Stochasticity: Morpion Solitaire is fully deterministic.

Terminal states: The episode ends when there are no available actions.

Initial states: Fig. 8 visualizes the initial configuration.

Notes: Morpion Solitaire is also known as Join Five. The current record for the disjoint variant lies at 82 and 178 for the joint version [28].

I. MULTI-ARMED BANDIT

Description: Multi-armed bandit is a 1-step MDP and is named after slot machines, which are commonly referred to as 1-armed bandits. Each arm yields a reward that is sampled from a Gaussian.

Action space: In an n -armed bandit, the agent has access to n actions, each corresponding to pulling one arm.

Rewards: Each arm is assigned a Gaussian from which the immediate reward is sampled.

Stochasticity: The only stochasticity is part of the reward function.

Terminal states: The episode always ends after 1 step.

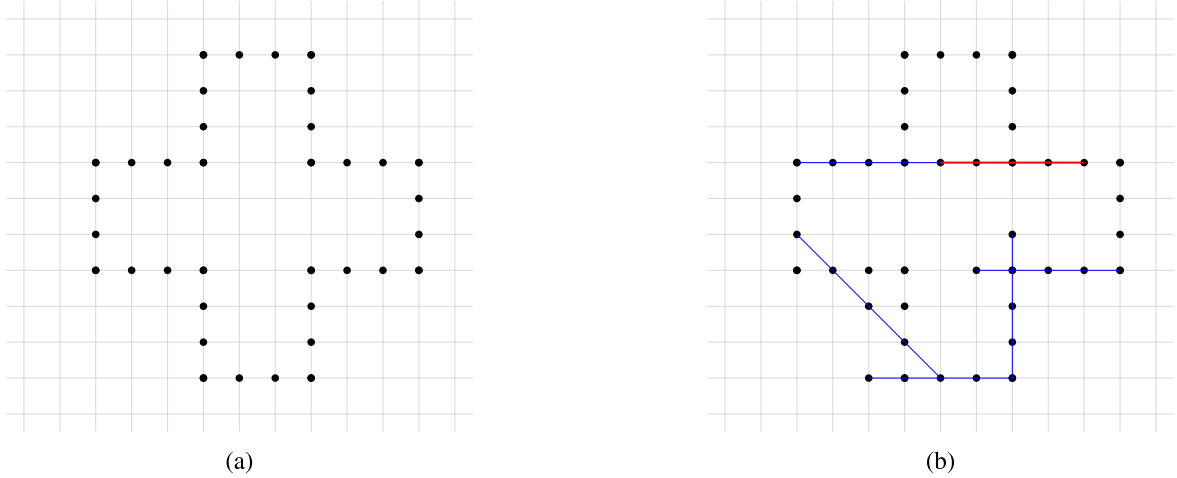
Notes: Note that strictly speaking, Multi-armed bandit does not fit into our MDP definition, which only encompasses non-stochastic reward functions.

J. NAVIGATION

Description: The objective is to navigate a robot across an $n \times m$ grid from the starting position at $(n, 1)$ to the goal at

TABLE 19. Manufacturer model statistics.

Instance	V^*	V_{mdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	-44.89	-756.84	2.96	42.77	50.00	1764
(2_IPPC)	N/A	-3507.90	3.76	N/A	50.00	12951
(3_IPPC)	N/A	-6188.82	4.64	N/A	50.00	1209480
(4_IPPC)	N/A	-2382.13	4.51	N/A	50.00	442559
(5_IPPC)	N/A	-5686.86	5.12	N/A	50.00	1773394
(6_IPPC)	N/A	-3870.84	4.66	N/A	50.00	198872
(7_IPPC)	N/A	-6821.13	5.67	N/A	50.00	1990417
(8_IPPC)	N/A	-3459.39	5.07	N/A	50.00	954264
(9_IPPC)	N/A	-6744.44	6.00	N/A	50.00	3054295
(10_IPPC)	N/A	-9942.88	21.78	N/A	50.00	2358223
(11_IPPC)	N/A	-24205.51	239.46	N/A	50.00	4827118

**FIGURE 8.** Two game states of Morpion Solitaire. The left image (a) shows the initial state, and the right image (b) shows a game state after five moves (if considering only the blue lines). The red line is a move that is only possible in the joint version of the game.**TABLE 20.** Manufacturer abstraction statistics.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC)	0.10	0.01	0.08	0.01	0.08	0.01

(n, m) in as few steps as possible. The robot can move to any of its four adjacent cells (up, down, left, right). However, each tile has an associated probability of resetting the robot back to the starting position $(n, 1)$ upon entry.

Action space: At most 4 actions corresponding to moving to any of the neighboring squares. Some actions might not be available if they would move the robot off the grid. If the idle parameter is set, the agent may additionally idle i.e., not move at all.

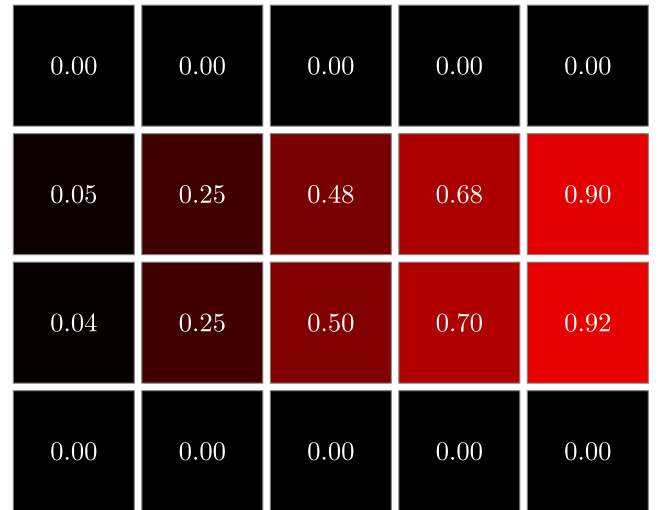
Rewards: A reward of -1 at each step is incurred except when the goal is reached, in which case a reward of 0 is dealt.

Stochasticity: At each step, the agent is either reset back to the starting position or the movement action succeeds. The reset probability is given by the probability attached to the tile the robot tries to move to.

Terminal states: The episode ends prematurely if the goal state is reached.

Initial states: The agent is initially placed at position $(n, 1)$.

Notes: Navigation was a test problem for the IPPC 2011. We deviated slightly from the PYRDDL problem in that

**FIGURE 9.** A visualization of the IPPC instance 3_Anand. The numbers in each square indicate its reset probability.

we allow for an early episode termination. The PYRDDL version allows idling. In total, this project comes with 10 pre-implemented instances.

K. PUSH YOUR LUCK

Description: In Push Your Luck, the agent has to decide which of n, m -sided, not-necessarily fair dice to roll or to

TABLE 21. Morpion Solitaire parameters.

Parameter	Description
Joint	Whether parallel lines may touch at their start/end points
Decoupled Action Space	Whether the action space is a two-step process

TABLE 22. Morpion Solitaire model statistics for the non-decoupled action space.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(joint)	N/A	53.58	11.89	1.00	53.58	4886484
(disjoint)	N/A	40.13	11.12	1.00	40.13	3457224

TABLE 23. Multi-armed bandit parameters.

Parameter	Description
Arm distributions	Specified as a sequence of means and standard deviations of the arms.

TABLE 24. Multi-armed bandit statistics for any Multi-armed bandit instance. The format is (μ, σ) where $\mu \in \mathbb{R}^n$ is the sequence of means and $\sigma \in (\mathbb{R}^+)^n$ is the sequence of standard deviations and n is the number of arms for a Multi-armed bandit instance.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(μ, σ)	$\max_{1 \leq i \leq n} \mu_i$	$\frac{1}{n} \cdot \sum_{i=1}^n \mu_i$	n	1	1	2

cash out. If cashed out, the agent receives a reward dependent on all dice faces that are marked. Faces are marked if they have been rolled (each face is shared by all n dice). However, if the agent rolls an already marked face or rolls two unmarked faces at the same time, all markings are removed.

Action space: At each step, the agent has to decide for each of the n dice whether to throw it or not. Throwing no dice corresponds to the cash out action. Hence, the number of legal actions in each state is equal to 2^n .

Rewards: The agent always receives a reward of 0 except when the agent cashes out. In that case, the reward is equal to

$$r = v(f_1) \cdot \dots \cdot v(f_k) \quad (6)$$

where $f_1, \dots, f_k$ are all faces that have previously been rolled, and $v : \{1, \dots, m\} \mapsto \mathbb{R}$ maps each die face to a reward-value.

Stochasticity: Each of the n dice has a unique probability distribution for its m faces. Each dice face is sampled independently.

Terminal states: There are no terminal states.

Initial states: Initially, no faces are marked.

Notes: Our Push Your Luck implementation does not calculate the transition probabilities for successors of state-action pairs as we deemed this to be computationally infeasible. Push Your Luck was used as a test problem for the IPPC 2018. In total, this project comes with 20 pre-implemented instances.

L. RACETRACK

Description: Racetrack involves navigating a car to a goal position on a graph in the fewest possible steps, starting from a randomly selected initial location. Each state is represented

TABLE 25. Navigation parameters.

Parameter	Description
Idle	Whether the agent is allowed to idle, i.e., do not move.

by a tuple in $\mathbb{Z}^2 \times \mathbb{Z}^2$, encoding the car's current position and velocity. At each step, the agent chooses an acceleration vector, applied independently along each axis. The velocity is updated by adding the acceleration, and the position is updated by adding the new velocity. If the car collides with an obstacle or boundary, and the reset parameter is enabled, it is returned to a random starting position with zero velocity. Otherwise, the velocity is set to zero, and the car is placed on the nearest empty tile along the straight line from the last valid position to the crash point.

Action space: The available actions are choosing an acceleration vector $a \in \{-1, 0, 1\}^2$.

Rewards: The agent incurs a constant reward of -1 .

Stochasticity: At each step, with some fixed probability, the chosen acceleration vector is replaced by $(0, 0)$.

Terminal states: If any goal state is reached, the episode terminates.

Initial states: Each instance defines a set of start cells. The agent is placed randomly at one of these start cells with uniform probability with 0 acceleration.

Notes: Racetrack originally introduced by Martin Gardner [29] and later formalized by Barto et al. [30]. Racetrack was used by Anand et al. [5] and Hostetler et al. [31]. In total, this project comes with 16 pre-implemented racetracks.

M. RED FINNED BLUE EYE

Description: In this environment, the agent is tasked with preserving and restoring the Red Finned Blue Eye (RFBE)

TABLE 26. Navigation model statistics when idling is not allowed.

Instance	V^*	V_{rndm}	Avg actions	Avg outcomes	Avg episode length	States
(1_Anand)	-6.41	-37.36	2.63	1.43	37.82	12
(2_Anand)	-7.67	-39.65	2.65	1.42	40.03	15
(3_Anand)	-10.52	-49.04	2.65	1.46	49.08	20
(4_Anand)	-12.77	-49.96	2.64	1.47	49.96	30
(1_IPPC)	-6.51	-38.54	2.62	1.43	38.96	12
(2_IPPC)	-7.47	-39.35	2.65	1.42	39.74	15
(3_IPPC)	-10.53	-49.15	2.64	1.46	49.19	20
(4_IPPC)	-13.06	-49.96	2.65	1.47	49.96	30
(5_IPPC)	-12.88	-42.03	2.57	1.42	42.33	30
(6_IPPC)	-20.71	-49.66	2.56	1.44	49.68	40
(7_IPPC)	-21.63	-49.99	2.56	1.44	49.99	50
(8_IPPC)	-15.96	-43.08	2.54	1.42	43.34	51
(9_IPPC)	-36.44	-49.87	2.52	1.43	49.88	61
(10_IPPC)	-40.95	-49.99	2.53	1.43	49.99	80

TABLE 27. Navigation abstraction statistics with idling not allowed.

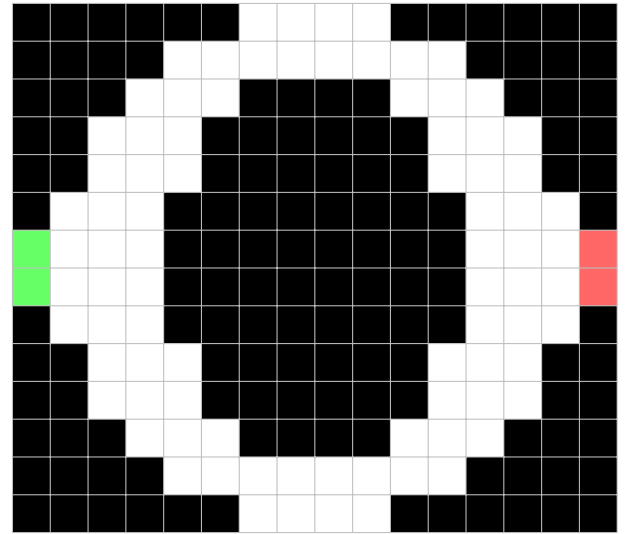
Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_Anand)	0.04	0.02	0.03	0.11	0.03	0.11
(2_Anand)	0.04	0.02	0.03	0.11	0.03	0.10
(3_Anand)	0.05	0.05	0.05	0.13	0.05	0.12
(4_Anand)	0.11	0.09	0.10	0.17	0.10	0.16
(1_IPPC)	0.04	0.02	0.03	0.11	0.03	0.11
(2_IPPC)	0.04	0.02	0.03	0.11	0.03	0.10
(3_IPPC)	0.05	0.05	0.05	0.13	0.05	0.12
(4_IPPC)	0.11	0.10	0.10	0.17	0.10	0.16
(5_IPPC)	0.02	0.02	0.02	0.09	0.02	0.08
(6_IPPC)	0.04	0.05	0.04	0.11	0.04	0.10
(7_IPPC)	0.09	0.13	0.09	0.16	0.09	0.15
(8_IPPC)	0.02	0.02	0.02	0.08	0.02	0.08
(9_IPPC)	0.04	0.05	0.05	0.11	0.05	0.10
(10_IPPC)	0.07	0.09	0.07	0.13	0.07	0.13

TABLE 28. Push your luck parameters.

Parameter	Description
File	Path to the model instance to be used.

fish population, which is being threatened by an invasive species of Gambusian fish. The ecosystem is being modelled as springs that are connected in a directed graph; however, the connections' accessibility is dependent on the current global water level, which changes stochastically. Gambusian spreads aggressively between connected springs.

Action space: Depending on the instance, the agent has a number of action points which can be spent on the following actions. At each step, the agent may poison a spring (which may kill both Gambusian and RFBE). This costs 1 action point and is only usable on springs that contain Gambusian fish. The agent may also manually remove Gambusian at a spring, which costs 2 action points, which is also only usable on springs with Gambusian fish. Lastly, the agent can relocate an RFBE population to a spring at a cost of 3 action points. The relocation action has a cooldown of 8 steps, and one can only relocate to an empty spring. The relocate action does not remove any existing RFBE but is only applicable if RFBE has not yet gone extinct (i.e., at least one spring still contains RFBE). Each spring can have at most one action targeting it.

**FIGURE 10.** A visualization of the Racetrack ring-2. Black cells indicate walls, white cells are track cells, the two green cells are the possible starting positions, and the two red cells are the goal positions.

Rewards: The reward at each step is given by

$$r = -30 \cdot n_{\text{poison}} - 100 \cdot b_{\text{extinct}} + 50 \cdot n_{\text{RFBE}} \quad (7)$$

where n_{poison} is the number of springs that have been poisoned in the current step, b_{extinct} is 0 if there is at least

TABLE 29. Push your luck model statistics.

Instance	V^*	V_{mdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	51.56	2.00	3.38	50.00	64
(2_IPPC)	N/A	60.99	2.00	5.35	50.00	1021
(3_IPPC)	N/A	78.20	2.00	10.30	50.00	41781
(4_IPPC)	N/A	40.53	2.00	3.38	50.00	64
(5_IPPC)	N/A	40.77	2.00	5.35	50.00	1021
(6_IPPC)	N/A	300.48	2.00	10.30	50.00	41781
(7_IPPC)	N/A	27.02	4.00	5.38	50.00	64
(8_IPPC)	N/A	40.83	4.00	13.86	50.00	1024
(9_IPPC)	N/A	3617.46	4.00	25.89	50.00	185017
(10_IPPC)	N/A	14.65	8.00	7.90	50.00	64
(11_IPPC)	N/A	18.29	8.00	24.93	50.00	1024
(12_IPPC)	N/A	4719.90	8.00	84.36	50.00	278372
(13_IPPC)	N/A	13.38	8.00	8.05	50.00	64
(14_IPPC)	N/A	17.00	8.00	25.35	50.00	1024
(15_IPPC)	N/A	180.10	8.00	58.09	50.00	106294
(16_IPPC)	N/A	64.08	16.00	76.55	50.00	91875
(17_IPPC)	N/A	22.48	32.00	82.92	50.00	67339
(18_IPPC)	N/A	9.91	32.00	81.32	50.00	37715
(19_IPPC)	N/A	3.04	32.00	117.43	50.00	346803
(20_IPPC)	N/A	2.72	32.00	114.08	50.00	193593

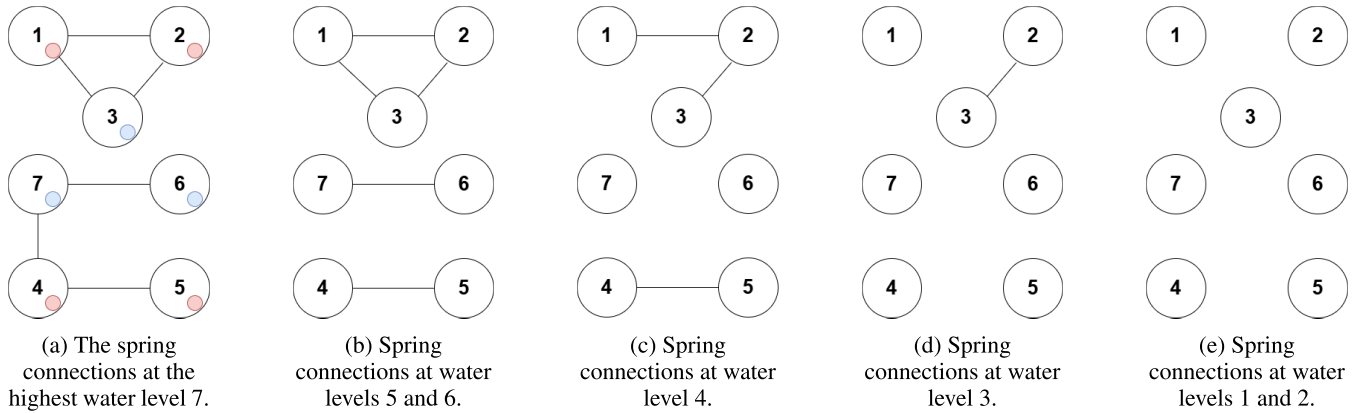
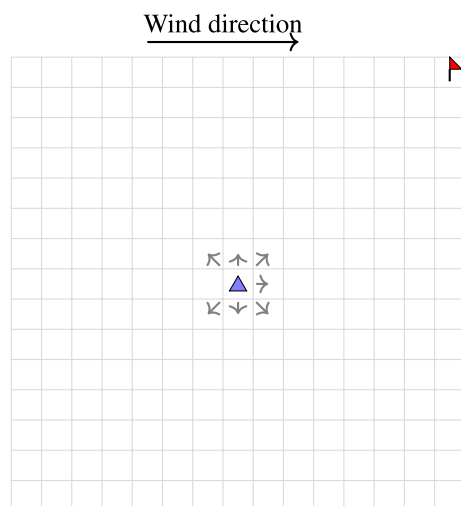


FIGURE 11. A visualization of the Red Finned Blue instance 1_IPPC, showing how the springs are connected as water level varies from 1 (least) to 7 (most). Furthermore, the leftmost subfigure (a) also shows the initial populations of the Gambusian (red) and the Red Finned Blue Eye fish (blue).

FIGURE 12. A visualization of the Sailing Wind using a grid size of 15×15 . The red flag indicates the goal position, the blue triangle is the agent's position, and the grey arrows indicate the direction it can move in in the current step.

one spring containing RFBE and 1 otherwise, and n_{RFBE} is the number of springs that contain RFBE.

Stochasticity: All probabilities referred to in the following are instance-specific. All actions targeting a single spring may randomly fail. Additionally, RFBE fish may randomly die out independently at each spring. The Gambusian spread is also stochastic and sampled as follows: Firstly, one of the seven water levels is sampled, then the Gambusian fish in each spring can randomly spread to any connected spring at the current water level. If the deterministic spread model parameter has been set, Gambusia spreads deterministically if there is a connection between two springs at the sampled water level. Otherwise, the spread between connected springs is random, with the spread probability being spring and water level dependent.

Terminal states: There are no terminal states.

Initial states: The initial fish populations are instance-dependent.

Notes: Red Finned Blue Eye has been a problem for the IPPC 2018 and is based on an ecology paper by

Nicol et al. [32]. The IPPC version uses the deterministic spread variant. Fig. 11 depicts the Red Finned Blue Eye instance 1_IPPC. In total, this project comes with 10 pre-implemented instances.

N. SAILING WIND

Description: Sailing Wind is a navigation task where the objective is to steer a ship from the starting position at $(1, 1)$ to the destination at (n, m) on an $n \times m$ grid while minimizing total cost. The cost of movement depends on the current wind direction, which changes stochastically at each time step, independent of the agent's actions.

Action space: The agent may move to any of the 8-neighboring squares. However, the agent is not allowed to move directly opposite to the wind direction. Also, actions that would move the agent outside the grid are not allowed either.

Rewards: The reward r per step is equal to $-1 + r(w, s)$ where

$$a \in W := \{(x, y) \mid x, y \in \{-1, 0, 1\}, (x, y) \neq (0, 0)\} \quad (8)$$

is the chosen action and $w \in W$ is the current wind direction, and $r(w, a) = ||w - a||_1$ is the Manhattan distance between w and a .

Stochasticity: At each step, the wind direction $w \in W$ stochastically changes to some $w' \in W$ with $||w - w'||_1 \leq 1$.

Terminal states: The episode ends prematurely if the goal state is reached.

Initial states: Initially, the agent is positioned at cell $(1, 1)$ and the wind initially blows in the direction $(0, 1)$.

Notes: Sailing Wind was originally proposed by Vanderbei [33]. The version described here is the one implemented by Anand et al. [5]. Sailing Wind was used as a test problem by Jiang et al. [6], Anand et al. [4], and Hostetler et al. [31]. Fig. 12 visualizes a state of a 15×15 Sailing Wind instance.

O. SAVING

Description: The agent is tasked with finding a good stock investment strategy. The agent has to decide whether to save its money, borrow money, invest, or sell its stock.

Action space: At each step, the agent has access to the save action. If the agent has not chosen the borrow action in t_{borrow} steps, the agent can also choose the borrow action. If t_{invest} steps have passed since the last invest action, the action may also choose the invest action. If less than t_{invest} steps have passed since the last investment, the agent may additionally choose to sell.

Rewards: The agent receives a reward of 1 if the save action has been chosen, a reward of 2 if borrow has been chosen, a reward of 0 if invest has been chosen, and a reward equal to the current stock price if sale has been chosen. If t_{borrow} steps have passed since the last borrow action has been chosen, the agent receives a reward of -3 .

Stochasticity: At each step, the stock price transitions to some integer value in $[g_{\min}, g_{\max}] \cap \mathbb{N}$ with uniform probability.

TABLE 30. Racetrack parameters.

Parameter	Description
File	Path to the model instance to be used.
Reset	Whether the agent is reset to the start in case of a crash.
Fail probability	The probability that the acceleration vector is set to $(0, 0)$.

Terminal states: There are no terminal states.

Initial states: The initial price is chosen randomly.

Notes: Saving was proposed as a toy problem by Hostetler et al. [9].

P. SKILL TEACHING

Description: In Skill Teaching, the agent takes the role of a tutor that is tasked with increasing the proficiency level of a student at various skills. The student can have one of three proficiency levels at each skill: Low, medium, and high. The skills from a prerequisite graph, giving the student higher chances of learning a new skill, the higher the prerequisites' levels of proficiency. Difficulty arises from the proficiency levels decaying if the corresponding skill is not regularly practised. This decay is deterministic for skills at medium proficiency and stochastic for those at high proficiency.

Action space: The agent may choose a skill at which to ask the student a question or give a hint. Hence, if there are c courses, there are $2c$ actions at each state. Giving a hint, deterministically, brings the student to the medium proficiency level if all prerequisites are of high proficiency. If asking a question and the student answers right, then this also brings a skill to the medium level without requiring high-level prerequisites. If the proficiency was already medium, then the student attains a high proficiency. Both hints and answering questions right prevent the proficiency level from decaying; however, answering wrong makes the student lose its high proficiency level.

If the idle parameter has been set, the agent may also idle. Furthermore, in the original RDDDL version, to simulate the student and teacher taking turns, the teacher's action did not have any effect every second step. By toggling the reduced action space parameter, the action space is reduced to a single idle action during the student's turn.

Rewards: Each skill has an instance-specific weight attached to it. The immediate reward is action independent and given by the weighted sum of skills that are at the high proficiency level minus the weighted sum of skills that are at the low proficiency level.

Stochasticity: For each skill at the highest level of proficiency, there is an instance and skill-dependent probability that the sufficiency decays to the medium level. Whether the student answered the question right that the teacher gave him is also random and given by an instance and skill-specific probability that is dependent on the following cases.

- 1) The skill is at a high level of proficiency.
- 2) The skill is at a medium level of proficiency and all prerequisites have a high proficiency level.

TABLE 31. Racetrack model statistics. Each instance has the format (File,Reset,Fail probability).

Instance	V^*	V_{ndm}	Avg actions	Avg outcomes	Avg episode length	States
(barto-big,0,0.0)	-17.00	-50.00	9.00	1.00	50.00	13705
(barto-small,0,0.0)	-10.00	-49.55	9.00	1.00	49.55	8889
(ring-1,0,0.0)	-5.00	-43.83	9.00	1.00	43.83	428
(ring-2,0,0.0)	-8.00	-49.01	9.00	1.00	49.01	1294
(barto-big,1,0.0)	-17.00	-50.00	9.00	2.69	50.00	5935
(barto-small,1,0.0)	-10.0	-49.99	9.00	2.15	49.99	6521
(ring-1,1,0.0)	-5.00	-49.56	9.00	1.41	49.56	403
(ring-2,1,0.0)	-8.00	-49.98	9.00	1.40	49.98	963
(barto-big,0,0.1)	-20.40	-50.00	9.00	1.68	50.00	13219
(barto-small,0,0.1)	-13.32	-49.62	9.00	1.65	49.62	8755
(ring-1,0,0.1)	-5.36	-44.26	9.00	1.54	44.26	428
(ring-2,0,0.1)	-8.32	-49.11	9.00	1.58	49.11	1293
(barto-big,1,0.1)	-22.44	-50.00	9.00	3.49	50.00	5681
(barto-small,1,0.1)	-13.32	-49.99	9.00	2.85	49.99	6437
(ring-1,1,0.1)	-5.47	-49.56	9.00	2.08	49.56	401
(ring-2,1,0.1)	-8.85	-49.99	9.00	2.01	49.99	966
(barto-big,0,0.2)	-22.52	-50.00	9.00	1.67	50.00	12478
(barto-small,0,0.2)	-12.32	-49.67	9.00	1.64	49.67	8557
(ring-1,0,0.2)	-5.85	-44.73	9.00	1.54	44.73	428
(ring-2,0,0.2)	-8.86	-49.22	9.00	1.58	49.22	1291
(barto-big,1,0.2)	-26.52	-50.00	9.00	3.49	50.00	5725
(barto-small,1,0.2)	15.38	-49.99	9.00	2.84	49.99	6322
(ring-1,1,0.2)	-6.28	-49.58	9.00	2.07	49.58	399
(ring-2,1,0.2)	-10.48	-49.98	9.00	2.00	49.98	933

TABLE 32. Racetrack abstraction statistics. Each instance has the format (File,Reset,Fail probability).

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(ring-1,0,0.0)	0.36	0.53	0.36	0.42	0.33	0.36
(ring-2,0,0.0)	0.38	0.53	0.37	0.42	0.34	0.37
(barto-small,0,0.0)	0.41	0.53	0.41	0.46	0.40	0.43
(barto-big,0,0.0)	0.52	0.61	0.52	0.55	0.51	0.53
(ring-1,1,0.0)	0.61	0.65	0.57	0.63	0.56	0.62
(ring-2,1,0.0)	0.53	0.58	0.47	0.53	0.47	0.53
(barto-small,1,0.0)	0.58	0.60	0.54	0.56	0.52	0.54
(barto-big,1,0.0)	0.63	0.64	0.59	0.60	0.57	0.58
(ring-1,0,0.1)	0.07	0.32	0.08	0.15	0.08	0.11
(ring-2,0,0.1)	0.13	0.34	0.13	0.20	0.12	0.15
(barto-big,0,0.1)	0.30	0.43	0.31	0.36	0.31	0.33
(barto-small,0,0.1)	0.16	0.33	0.16	0.23	0.16	0.19
(ring-1,1,0.1)	0.24	0.30	0.23	0.23	0.23	0.23
(ring-2,1,0.1)	0.37	0.43	0.32	0.34	0.32	0.33
(barto-small,1,0.1)	0.20	0.35	0.20	0.25	0.20	0.24
(ring-1,0,0.2)	0.07	0.31	0.08	0.16	0.08	0.12
(ring-2,0,0.2)	0.13	0.34	0.13	0.21	0.12	0.16
(barto-small,0,0.2)	0.16	0.33	0.16	0.23	0.16	0.20
(barto-big,1,0.1)	0.34	0.45	0.34	0.37	0.34	0.36
(barto-big,0,0.2)	0.31	0.43	0.31	0.36	0.31	0.34
(ring-1,1,0.2)	0.24	0.30	0.24	0.23	0.25	0.23
(ring-2,1,0.2)	0.37	0.43	0.33	0.34	0.32	0.33
(barto-small,1,0.2)	0.20	0.35	0.20	0.24	0.20	0.23
(barto-big,1,0.2)	0.34	0.44	0.34	0.37	0.34	0.36

TABLE 33. Red finned blue eye parameters.

Parameter	Description
File	Path to the model instance to be used.
Deterministic spread	Whether Gambusia spreads deterministically.

- 3) The skill is at a medium level of proficiency, but not all prerequisites have a high proficiency level. In this case, the answered-right probability is proportional to the number of prerequisites.
- 4) The skill is at a low proficiency level but all prerequisites are at a high proficiency level. In this case, the

answered-right probability is also proportional to the number of prerequisites.

Terminal states: There are no terminal states.

Initial states: Initially, all students' skills are at the lowest proficiency level.

Notes: Skill Teaching has been a problem for the IPPC 2014. The IPPC version does not have the above-described reduced action space but allows idling. Fig. 13 shows the Skill Teaching prerequisites graph for instance 5_IPPC. In total, this project comes with 10 pre-implemented instances.

TABLE 34. Red finned blue eye model statistics. Each instance has the format (File,Deterministic spread).

Instance	V^*	V_{mdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC,0)	N/A	-1146.89	4.69	N/A	50.00	5407
(2_IPPC,0)	N/A	92.01	5.24	N/A	50.00	19304
(3_IPPC,0)	N/A	-4434.91	15.48	N/A	50.00	29756
(4_IPPC,0)	N/A	-1460.02	4.17	N/A	50.00	23929
(5_IPPC,0)	N/A	-3681.47	27.09	N/A	50.00	16443
(6_IPPC,0)	N/A	-1693.36	23.79	N/A	50.00	222062
(7_IPPC,0)	N/A	6235.79	3.44	N/A	50.00	114892
(8_IPPC,0)	N/A	-2737.12	41.18	N/A	50.00	261662
(9_IPPC,0)	N/A	-2198.65	20.14	N/A	50.00	127521
(10_IPPC,0)	N/A	-1297.10	321.45	N/A	50.00	196814
(1_IPPC,1)	N/A	-6810.63	31.04	8.57	50.00	5207
(2_IPPC,1)	N/A	-5368.44	57.16	16.39	50.00	16190
(3_IPPC,1)	N/A	-11273.97	94.92	10.96	50.00	24929
(4_IPPC,1)	N/A	-10294.59	51.42	N/A	50.00	10358
(5_IPPC,1)	N/A	-9462.54	484.84	N/A	50.00	17212
(6_IPPC,1)	N/A	-12258.25	1001.81	N/A	50.00	64589
(7_IPPC,1)	N/A	2803.84	425.21	N/A	50.00	213108
(8_IPPC,1)	N/A	-11920.34	1386.59	N/A	50.00	42027
(9_IPPC,1)	N/A	-11442.77	766.66	N/A	50.00	83600
(10_IPPC,1)	N/A	-4561.27	5422.55	N/A	50.00	33598

Q. SysAdmin

Description: In SysAdmin, the agent takes the role of a network manager and is tasked with keeping as many computers (organized in various topologies) running. Formally, the state space is $\{0, 1\}^n$ (describing which machines are currently operating), which can be influenced by rebooting at most one machine per step.

Action space: The agent can reboot at most one machine per step, which causes it to deterministically work in the next step.

Rewards: The reward $r = r_t + r_a$ can be decoupled into an action-independent component r_t equal to the number of currently running machines and a state-independent component r_a equal to -0.75 if a machine has been rebooted or 0 if the idle action has been chosen.

Stochasticity: Each currently operating machine that was not chosen as the action can randomly and independently fail in the next step. The machines form a directed dependency graph, and the fail probability increases linearly with the number of neighboring non-operating machines. The slope is approximately inversely proportional to the number of neighbors. However, it is also possible for each currently non-operating machine to reboot itself with an instance-dependent probability p_{reboot} .

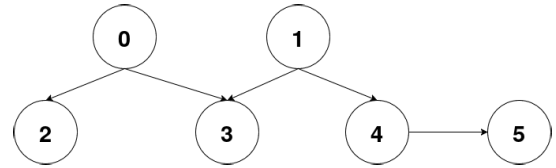
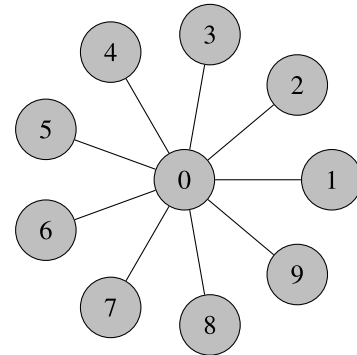
Terminal states: There are no terminal states.

Initial states: Initially, all machines are running.

Notes: SysAdmin was proposed by Guestrin et al. [34] and used for the IPPC 2011. Fig. 14 visualizes the SysAdmin instance 4_IPPC. In total, this project comes with 6 pre-implemented instances.

R. TAMARISK

Description: Tamarisk models the expansion of an invasive plant in a river system which is composed of a chain of reaches where each reach contains a number of slots that may be unoccupied, occupied by a native plant, occupied by

**FIGURE 13.** The prerequisites graph for the Skill Teaching instance 5_IPPC.**FIGURE 14.** Visualization of the SysAdmin topology of instance 4_IPPC.

the invasive Tamarisk plant, or occupied by both. The spread of both plant types is stochastic, with downstream-spread having a higher probability than upstream spread. The agent has to decide where and when to eradicate Tamarisk or restore native plants whilst balancing the action's costs with the penalties incurred for existing Tamarisk plants.

Action space: At each time step, the agent may do nothing, eradicate Tamarisk at a slot, or restore native plants at a slot. The action is applied to all slots in that reach.

Rewards: The reward per step is given by $r = r_a - 5 \cdot n_{\text{invaded}} - 0.5 \cdot n_{\text{tree}} - 0.25 \cdot n_{\text{empty}}$ where $r_a = 0.9 - 0.4 \cdot n_{\text{empty_slot}}$ if the restoration action has been chosen and $r_a = 0.49$ for the eradication action, $n_{\text{empty_slot}}$ is the number of empty slots at the chosen reach, n_{invaded} is the number of

TABLE 35. Sailing Wind parameters.

Parameter	Description
$n \in \mathbb{N}$	Number of grid rows.
$m \in \mathbb{N}$	Number of grid columns.

reaches that contain at least one slot occupied by Tamarisk, n_{tree} is the number of slots occupied by Tamarisk, and n_{empty} is the number of empty slots (i.e. no native plant or Tamarisk).

Stochasticity: The following stochastic events occur independently at each slot.

- 1) If there are both Tamarisk and native plants at a slot, Tamarisk may randomly die out.
- 2) If a slot contains Tamarisk and the eradicate action has been applied to the slot's reach, then Tamarisk may randomly survive.
- 3) If a slot contains Tamarisk but no native plants, Tamarisk dies out randomly.
- 4) If a slot contains neither Tamarisk nor native plants, then Tamarisk may spawn with a chance that increases with the number of Tamarisk at the slots of the same reach, the downstream, or upstream reach.
- 5) If there are both Tamarisk and native plants at a slot, the native plants may die out randomly.
- 6) If a slot contains no plants and the restore action has been applied to the slot's reach, then native plants may randomly fail to be restored.
- 7) If a slot contains native plants but no Tamarisk, then the native plants may randomly die out.
- 8) If a slot contains no plants, native plants may randomly spawn.

Terminal states: There are no terminal states.

Initial states: The initial Tamarisk and native population is instance-dependent.

Notes: Tamarisk was a test problem for the IPPC 2014. Tamarisk was adapted from a reinforcement learning competition [35], which themselves based their problem on a model from theoretical biology [36]. In total, this project comes with 10 pre-implemented instances.

S. TOY SOCCER

Description: In Toy Soccer, the only decision the agent has to make is whether to shoot or pass a ball. The domain consists of only five states: The initial state, the goal state, the ball being at the left winger, or the ball being at one of the right wingers.

Action space: At the initial state, the agent has two available actions: Passing left or right, or shooting directly. In all remaining states, non-terminal states, the agent only has a single action, which is to shoot.

Rewards: The agent always receives a reward of 0, hence all policies are optimal.

Stochasticity: When the agent decides to pass right, the ball arrives at either winger with uniform probability.

Terminal states: The episode terminates when the goal state is reached. Hence, an episode has a maximum length of 2.

Notes: Toy Soccer was introduced by Anand et al. [4] to compare different abstraction frameworks.

T. TRAFFIC

Description: In this environment, the agent is tasked with simultaneously controlling a number of traffic lights with the goal of minimizing traffic jams. This traffic is modelled as a directed graph; however, some edges are only available depending on the state of a traffic light. Each vertex may either contain a car or not.

Action space: At each step, the agent has to decide for each of the t traffic lights whether to advance its state or not. Hence, the action space is of size 2^t .

Rewards: The reward is equal to the negative of the number of traffic jams of the current state. A traffic jam is defined as any pair of connected cells (or vertices) that are both occupied.

Stochasticity: The only stochasticity comes from random arrivals at dedicated perimeter cells. At each of these cells, there is an instance and cell-dependent chance of a new car occupying that cell if it was not already.

Terminal states: There are no terminal states.

Initial states: Each traffic light starts at the same state. The initially occupied cells are instance-specific.

Notes: Traffic has been a problem for the IPPC 2014 and is based on cell transition models [37], [38]. Fig. 15 visualizes the Traffic instance 1_IPPC. In total, this project comes with 10 pre-implemented instances.

U. TRIANGLE TIREWORLD

Description: The agent is a car that traverses a graph. At each step, the car may move to an adjacent node, change its tire, or load a tire. The goal is to reach a designated goal node. At each step, the car's tire may randomly break. The agent cannot move with a broken tire. If available, a spare tire (at most one can be carried) can be used to replace the current tire. Some nodes contain spare tires, which, when the agent visits them, can be picked up.

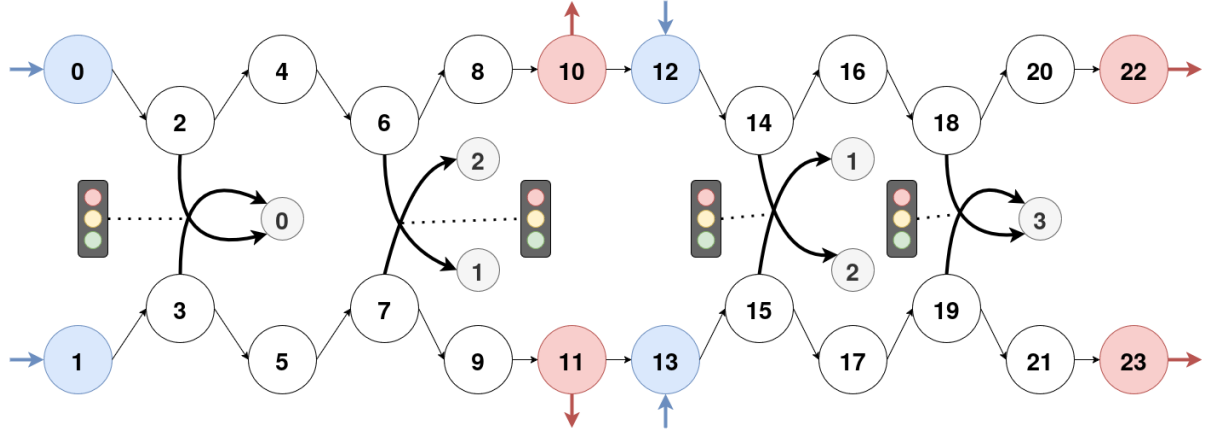
Action space: At each step, the agent may load a tire, which has no effect if there is no spare tire, or change its tire. This action also has no effect if the agent does not carry a previously loaded tire. If neither of these two actions is chosen, the agent may also idle (if the idling parameter is set) or move to any of the neighboring nodes. If the reduced action space parameter is not set, the agent may also attempt to move to any vertex; however, this action will have no effect if the target is not a neighboring vertex.

Rewards: The agent always receives a reward of -1 except when it reaches the goal state, which yields a reward of 100.

Stochasticity: At each step, there is a constant probability that the agent's tire may break.

TABLE 36. Sailing Wind model statistics. Each instance is denoted by its size, i.e. we use $n = m$.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(10)	-21.47	-134.28	6.12	3.00	49.31	800
(15)	-35.06	-135.43	6.27	3.00	49.98	1800
(20)	-48.82	-135.32	6.30	3.00	50.00	3159

**FIGURE 15.** The traffic graph of the Traffic instance 1_IPPC. Blue cells are cells that receive a random car inflow. Cells marked in red are cells where the traffic flow exits, and the bold edges are intersection edges. Only one of crossing bold edges can ever be active at the same time.**TABLE 37.** Sailing Wind abstraction statistics. Each instance is denoted by its size, i.e. we use $n = m$.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(10)	0.04	0.01	0.03	0.01	0.03	0.01
(15)	0.06	0.01	0.06	0.02	0.06	0.02
(20)	0.08	0.01	0.07	0.02	0.07	0.02

TABLE 38. Saving parameters.

Parameter	Description
$g_{\min} \in \mathbb{Z}$	Minimal price
$g_{\max} \in \mathbb{Z}$	Maximal price
$t_{\text{borrow}} \in \mathbb{N}$	Number of steps after borrow after which the agent has to repay
$t_{\text{invest}} \in \mathbb{N}$	Number of steps after invest after which agent can sell its stock

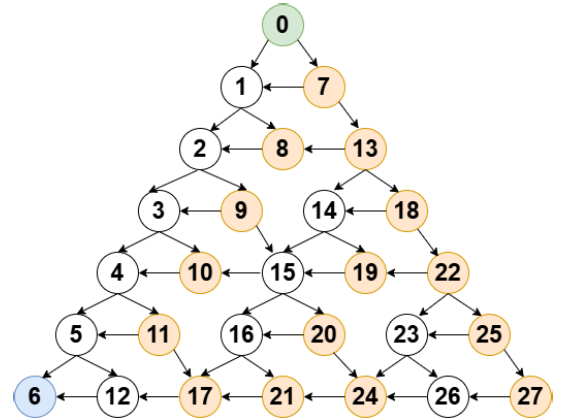
Terminal states: The episode terminates when the goal state is reached.

Initial states: The agent starts with a non-flat tire and with no spare tire. The initial position is instance-dependent.

Notes: Triangle Tireworld was a test problem for the IPPC 2014 and originally proposed by Little and Thiébaux [39]. The PYRDDL version allows idling but does not use the reduced action space option. Fig. 16 visualizes a concrete Triangle Tireworld instance. In total, this project comes with 10 pre-implemented instances. The key planning challenge in Triangle Tireworld is that the shortest path to the goal is not the optimal route because it does not contain any tires.

V. WILDFIRE

Description: Wildfire models the spread of a fire on a grid. Each grid cell is either untouched, burning, or out-

**FIGURE 16.** A visualization of the Triangle Tireworld instance 5_IPPC. The red node is the goal, green is the start, and yellow nodes are those with a spare tire. An edge from node X to node Y indicates that the agent can move from X to Y.

of-fuel, meaning that no new fire can ignite at this cell. If a cell is untouched, it can at each time step randomly ignite with the probability increasing exponentially in the number of neighboring burning cells. The neighborhood is defined on an instance level with most instances choosing the 8-neighborhood and manually cutting a handful of neighborhood connections between individual cells. The agent is tasked with controlling the spread of the fire.

Action space: The agent's actions are to idle, to cut out the fuel of a cell, or to put out a fire at any cell.

Rewards: The rewards $r = r_a + r_t$ can be decoupled into an action-only dependent component r_a and a state-only

TABLE 39. Saving model statistics. Each instance has the format $(g_{\min}, g_{\max}, t_{\text{borrow}}, t_{\text{invest}})$.

Instance	V^*	V_{mdm}	Avg actions	Avg outcomes	Avg episode length	States
(-3,3,3,3)	51.00	13.76	2.51	7.00	50.00	3060
(-3,3,4,4)	52.45	15.67	2.44	7.00	50.00	3318
(-3,3,5,5)	53.72	17.12	2.39	7.00	50.00	3628
(-4,4,3,3)	57.33	13.76	2.51	9.00	50.00	3798
(-4,4,4,4)	59.63	15.67	2.44	9.00	50.00	4112
(-4,4,5,5)	61.01	17.11	2.39	9.00	50.00	4518
(-5,5,3,3)	64.76	13.76	2.51	11.00	50.00	4523
(-5,5,4,4)	67.31	15.66	2.44	11.00	50.00	4914
(-5,5,5,5)	68.80	17.10	2.39	11.00	50.00	5379

TABLE 40. Saving abstraction statistics. Each instance has the format $(g_{\min}, g_{\max}, t_{\text{borrow}}, t_{\text{invest}})$.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(-3,3,3,3)	0.36	0.02	0.28	0.03	0.26	0.03
(-3,3,4,4)	0.32	0.02	0.24	0.03	0.22	0.03
(-3,3,5,5)	0.32	0.01	0.24	0.03	0.23	0.03
(-4,4,3,3)	0.32	0.01	0.25	0.02	0.22	0.02
(-4,4,4,4)	0.33	0.01	0.25	0.02	0.22	0.02
(-4,4,5,5)	0.32	0.01	0.25	0.02	0.23	0.02
(-5,5,3,3)	0.33	0.01	0.25	0.02	0.22	0.02
(-5,5,4,4)	0.32	0.01	0.24	0.02	0.22	0.02
(-5,5,5,5)	0.32	0.01	0.24	0.02	0.23	0.02

dependent component r_t . The former is equal to -5 for the cutout action, -10 for the putput action, and 0 for idling. The latter, r_t is equal

$$r_t = -100 \cdot n_{t_{\text{burn}}} - 5 \cdot n_{nt_{\text{burn}}} \quad (9)$$

where $n_{t_{\text{burn}}}$ is the number of target cells that are burning and $n_{nt_{\text{burn}}}$ is the number of non-target cells that are burning.

Stochasticity: Each non-target cell that is neither out-of-fuel and neither burning can ignite with probability p_{ignite} that is equal to

$$p_{\text{ignite}} = \frac{1}{1 + e^{4.5 - n_{\text{burn_neighbor}}}} \quad (10)$$

where $n_{\text{burn_neighbor}}$ is the number of burning neighboring cells. Target cells have the same ignition probability except when a target cell has no burning neighbors; in that case, a target cell cannot spontaneously ignite.

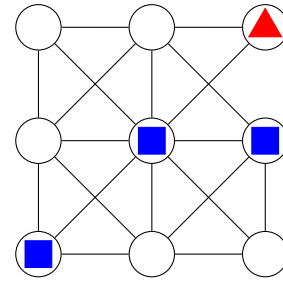
Terminal states: There are no terminal states.

Initial states: Initially, no cell is out of fuel, however, each instance specifies a set of initially burning cells.

Notes: Wildfire was used for the IPPC 2014 [25] and is based of the work by Karafyllidis and Thanailakis [40]. Fig. 17 visualizes the initial state of the Wildfire instance 1_IPPC. In total, this project comes with 10 pre-implemented instances. Wildfire is challenging because extremely rare events (e.g. a fire spawning randomly) have catastrophic consequences and thus have to be considered during decision-making.

W. WILDLIFE PRESERVE

Description: In Wildlife Preserve, the agent manages rangers to defend areas from poachers. If a ranger is sent to defend

**FIGURE 17.** A visualization of the initial state of the Wildfire instance 1_IPPC. The red triangle indicates fire and the blue squares indicate so-called targets which have to be protected.

an area, a poacher decides to attack, the poacher is caught and can not attack an area in the next step. Each poacher has different area preferences and remembers how often each area was defended in the last couple of steps.

Action space: The agent has to decide for each of its rangers which area they have to defend. Sending multiple rangers to a single area has no additional effect than what a single ranger would have.

Rewards: Each area a is assigned an instance-dependent defense reward $r_{\text{def},a}$ and an attack penalty $r_{\text{atk},a}$. The immediate reward is given by

$$\sum_{a \in A_{\text{def}}} r_{\text{def},a} + \sum_{a \in A_{\text{atk}}} r_{\text{atk},a} \quad (11)$$

where A_{def} is the set of areas that were successfully defended in the last step (i.e., they were defended by a ranger or not attacked by a poacher) and A_{atk} is the number of areas that were successfully attacked (i.e. not defended and attacked by a poacher).

Stochasticity: Each poacher that has not been caught in the last step randomly attacks an area. Each poacher assigns a weight to each area, which is instance-specific and dependent on the number of times that area has been attacked in the last n steps (n is also instance-dependent) and the poacher's own area preference. The poacher then samples an area to attack with a probability proportional to the area's weight.

Terminal states: There are no terminal states.

Initial states: Initially, no poachers are caught, and each poacher remembers no defenses. It is assumed that there were no successful attacks in the (hypothetical) last step.

TABLE 41. Skill teaching parameters.

Parameter	Description
File	Path to the model instance to be used.
Idle	Whether the agent is allowed to idle.
Reduced action space	Whether the agent has only a single idle action during student's turn.

TABLE 42. Skill teaching model statistics for the reduced action space case with idling not being allowed.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	87.33	7.47	2.50	1.46	50.00	63
(2_IPPC)	101.10	2.44	2.50	1.45	50.00	63
(3_IPPC)	157.86	-274.47	4.50	1.42	50.00	934
(4_IPPC)	171.95	-306.13	4.50	1.42	50.00	923
(5_IPPC)	102.00	-547.24	6.50	1.35	50.00	4142
(6_IPPC)	126.41	-630.58	6.50	1.36	50.00	3965
(7_IPPC)	42.05	-787.07	7.50	1.35	50.00	6102
(8_IPPC)	-41.38	-946.51	7.50	1.34	50.00	5047
(9_IPPC)	N/A	-904.99	8.50	1.33	50.00	8331
(10_IPPC)	N/A	-1104.71	8.50	1.32	50.00	7170

TABLE 43. Skill teaching abstraction statistics for the reduced action space case with idling not being allowed.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC)	0.23	0.04	0.14	0.04	0.07	0.03
(2_IPPC)	0.22	0.04	0.13	0.04	0.06	0.02
(3_IPPC)	0.20	0.06	0.11	0.06	0.04	0.02
(4_IPPC)	0.21	0.06	0.12	0.06	0.04	0.02
(5_IPPC)	0.29	0.11	0.17	0.11	0.06	0.04
(6_IPPC)	0.30	0.11	0.18	0.11	0.07	0.05
(7_IPPC)	0.30	0.13	0.17	0.13	0.06	0.05
(8_IPPC)	0.35	0.14	0.21	0.13	0.09	0.06

Notes: Our Wildlife Preserve implementation does not contain transition probabilities for sampled successors of state-action-pairs. Wildlife Preserve has been a problem for the IPPC 2018 and is based on work by Fang et al. [41] and by Nguyen et al. [42]. In total, this project comes with 20 pre-implemented instances.

APPENDIX C

CATALOGUE OF TWO-PLAYER STOCHASTIC GAMES (SG)

In this section, we give a description of each Two-player game that we implemented. For each game, we first give a high-level overview of the environment, then we describe the action space, and the terminal states in more detail. If applicable, we provide a table of parameters to customize the environments. As with the MDPs, we ran a random agent on various parameter combinations for 100000 games with a maximal game length of 200 against another random agent as the opponent and we list the average return V_{rdm} , the average number of legal actions, the total number of encountered states, and the average episode length. For each game, the statistics are always out of the player's perspective who has the first turn. Hence, in Connect 4, for example, the average value is positive as the beginning player has a slight advantage.

TABLE 44. SysAdmin parameters.

Parameter	Description
File	Path to the model instance to be used.

In contrast to the MDPs, each of the following two-player games is deterministic. In all two-player games, a reward of 0 is dealt at each step, unless the game ends, then 1 is dealt to the winner and -1 to the loser, or 0 to both in case of a draw. If the zero-sum parameter (that all of the following games possess) is set to false, the reward could also be 1 in case of a win and 0 otherwise, for both a win or a loss. We refer to this case as binary rewards.

The two-player SGs that we implemented are either board games, strategy games from the Stratega framework [43], or invented by us.

A. CAPTURE THE FLAG

Description: Capture the Flag is played on a 2-dimensional grid. Each player controls an initially equal number of units. The goal is to capture the enemy's flag, initially located at its base (a designated cell) and bring it to one's own base. At each step, the player at turn controls one unit, which can be moved and optionally attack an enemy unit. Once the player has cycled through all units, the player's turn ends. Units can be moved and used to attack the opponent's units. Each unit attack deals 1 damage point, and all units have exactly 2 health points. Once a unit's health reaches 0, it respawns as close as possible to a designated respawn point of that unit. If the unit carried a flag, the flag is dropped at the point where the unit died. The opponent's flag is automatically picked up when a unit moves to the flag's cell.

Action space: An action controls exactly one unit. The player may move the unit if it does not move the unit out of bounds, bump into another unit, or cells marked as terrain.

TABLE 45. SysAdmin model statistics. These statistics exclude the IPPC instances 7-10 as they are not supported by our implementation since these feature more than 32 machines but we encode the machine stati with a 32-bit integer.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	427.00	260.63	11.00	558.82	50.00	1024
(2_IPPC)	387.32	196.18	11.00	558.82	50.00	1024
(3_IPPC)	N/A	406.08	21.00	N/A	50.00	611261
(4_IPPC)	N/A	363.30	21.00	N/A	50.00	565682
(5_IPPC)	N/A	509.40	31.00	N/A	50.00	2989938
(6_IPPC)	N/A	446.09	31.00	N/A	50.00	2509448

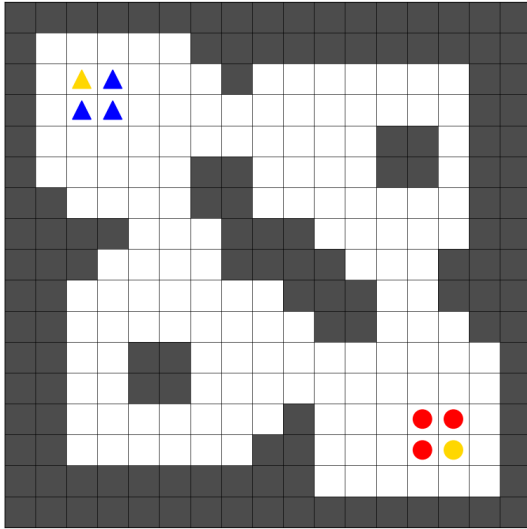


FIGURE 18. The initial unit position of Capture the Flag. Black squares indicate walls. Shapes of the same shape belong to the same player. Blue and red represent units, and yellow represents flags.

TABLE 46. SysAdmin abstraction statistics.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC)	0.01	0.03	0.01	0.01	0.01	0.01
(2_IPPC)	0.01	0.02	0.00	0.01	0.00	0.00

TABLE 47. Tamarisk parameters.

Parameter	Description
File	Path to the model instance to be used.

The unit can be moved to any cell whose x and y distances are less than two. Optionally, an action may indicate that after the movement, the unit may attack any opponent unit within the 8-neighborhood of the unit's final position.

Terminal states: The game ends when one player manages to carry the opponent's flag to its own base. This player wins. There are no draws.

Initial states: The initial unit configuration is visualized in Fig. 18

Notes: Captured the Flag was originally developed for the Stratega framework [43]. There is only a single pre-implemented Capture the Flag map.

B. SimpleChess

Description: SimpleChess is played on an 8×8 board. Each player starts with an identical set of 16 pieces of various types. Players take turns moving exactly one piece, whose movement is dependent on its type. Opponent pieces can be captured (removed from the board) by performing a movement action that ends at the tile the opponent piece occupies. The game ends when either player manages to capture the opponent's king, which is one of its pieces.

Action space: Per step, a player may move any of its pieces. The following lists how each piece type moves:

- 1) Pawn: Moves one square forward if unoccupied, or captures one square diagonally forward. Pawns also have the option to move two squares forward from their starting rank.
- 2) Rook: Moves any number of vacant cells vertically or horizontally that do not skip over any other piece.
- 3) Knight: Moves in an "L"-shape: two squares along a vertical or horizontal, and then one square perpendicular to that.
- 4) Bishop: Moves any number of vacant squares diagonally. Like the Rook, the Bishop may not skip over any other pieces.
- 5) Queen: Available movement is the union of the Bishop's and Rook's options.
- 6) King: Moves exactly one square in any direction (horizontally, vertically, or diagonally).

Terminal states: The game ends when a king is captured. The player whose king has been captured loses.

Notes: In contrast to standard Chess, SimpleChess does not implement castling, pawn promotions, en passant, or the 3-repetition rule.

Initial states: The initial unit configuration is visualized in Fig. 19.¹

C. CONNECT 4

Description: Connect 4 is played on a grid with 7 columns and 6 rows. Each turn, one player places a stone of their color in one of the columns that is not yet filled with stones. The stone occupies the first cell in the chosen column that is not yet occupied.

¹https://upload.wikimedia.org/wikipedia/commons/a/a7/Chess_board_blank.svg Beao, CC BY-SA 3.0 via Wikimedia Commons.

TABLE 48. Tamarisk model statistics.

Instance	V^*	V_{rndm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-850.91	9.00	N/A	50.00	32334
(2_IPPC)	N/A	-1174.10	9.00	N/A	50.00	287888
(3_IPPC)	N/A	-1157.38	11.00	N/A	50.00	154742
(4_IPPC)	N/A	-1478.31	11.00	N/A	50.00	645975
(5_IPPC)	N/A	-1610.85	13.00	N/A	50.00	264580
(6_IPPC)	N/A	-1797.21	13.00	N/A	50.00	1187384
(7_IPPC)	N/A	-1805.53	15.00	N/A	50.00	652999
(8_IPPC)	N/A	-2100.51	15.00	N/A	50.00	1691527
(9_IPPC)	N/A	-1966.07	17.00	N/A	50.00	1147991
(10_IPPC)	N/A	-2344.34	17.00	N/A	50.00	2663295

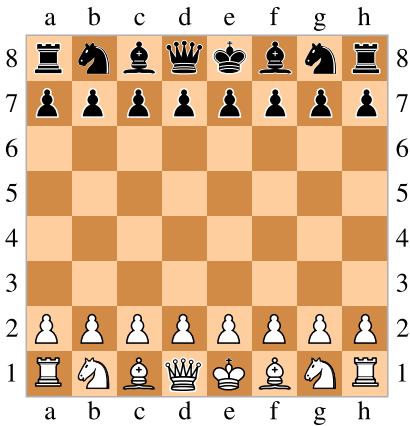


FIGURE 19. Visualization of the initial state of SimpleChess. The cell a2 is occupied by a pawn, a1 by a rook, b1 by a knight, c1 by a bishop, and d1 by the king.

TABLE 49. Traffic parameters.

Parameter	Description
File	Path to the model instance to be used.

Action space: The player at turn can choose any column that is not fully filled with stones as its action.

Terminal states: The game ends in a draw when every column is fully filled or when one player completes a vertical, horizontal, or diagonal line of its stones. In that case, the player wins.

Initial states: The board is initially empty.

Notes: Connect 4 is a well-known board game that is played in the real world. Fig. 20 visualizes a concrete Connect 4 game state.

D. CONSTRICTOR

Description: Constrictor is played on an n times n grid. Players take turns moving to any of the neighboring (4-neighborhood) grid cells that neither moves the player out of bounds nor hits any cell that has already been visited by either of the two players. The game ends when one player has nowhere left to move.

Action space: The player has to move to any of the 4 adjacent cells. Only those movement actions are legal that

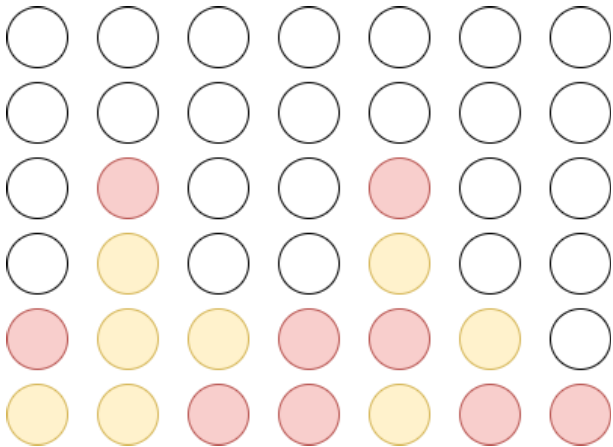


FIGURE 20. A visualization of a Connect 4 board state that could have occurred after 17 moves of playing.

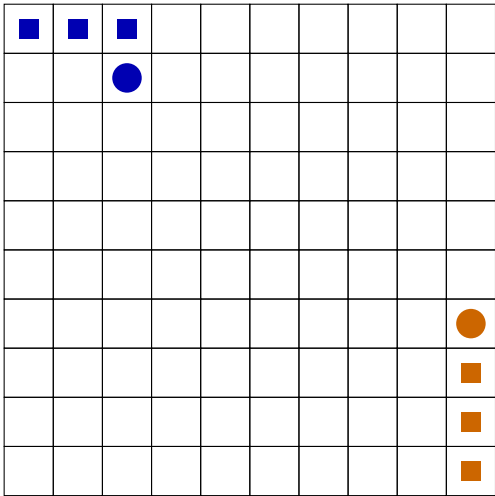


FIGURE 21. A visualization of a Constrictor game state using a 10 × 10 arena that could have occurred after 6 steps. The circles indicate the players' heads and the squares their bodies.

neither move the player out of bounds nor to a cell that has already been visited.

Terminal states: The game ends when one player has no legal actions left. This player loses, the other wins. A draw is not possible.

TABLE 50. Traffic model statistics.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-79.24	16.00	10.02	50.00	2581018
(2_IPPC)	N/A	-103.53	16.00	7.65	50.00	2205840
(3_IPPC)	N/A	-158.55	16.00	8.94	50.00	4545413
(4_IPPC)	N/A	-216.03	16.00	5.99	50.00	4449428
(5_IPPC)	N/A	-217.97	16.00	10.02	50.00	4612441
(6_IPPC)	N/A	-255.94	16.00	8.81	50.00	4655935
(7_IPPC)	N/A	-267.81	16.00	10.90	50.00	4648070
(8_IPPC)	N/A	-291.10	16.00	8.69	50.00	4700758
(9_IPPC)	N/A	-265.49	16.00	10.95	50.00	4703018
(10_IPPC)	N/A	-406.51	16.00	8.91	50.00	4692308

Initial states: The board is initially empty with the players being placed at opposite ends of the board.

Notes: Constrictor is a game mode of the Battlesnake AI competition [44] where the two players move simultaneously. Though the online version supports more than two players, we implemented only the two-player version. Fig. 21 visualizes a Constrictor game state.

E. KILL THE KING

Description: Kill the King is played on a 2-dimensional grid. Each player controls an initially equal number of units. The goal is to eliminate the enemy's king, which is a special, unique unit. At each step, the player at turn controls one unit, which can be moved and optionally execute its special attack. Once the player has cycled through all units, the player's turn ends. There are four types of units with the following properties:

Action space: An action controls exactly one unit. The player may move the unit to any of the 8 neighboring cells if it does not move the unit out of bounds, bump into another unit, or cells marked as terrain. Optionally, an action may indicate that after the movement, the unit's special ability is used, targeting any targetable unit within its ability range.

Terminal states: The game ends when one player's king dies. The player whose king died, loses. There are no draws.

Initial states: Though the map is the same, initially, one of 50 unit positions is sampled. Fig. 22 visualizes one such initial position.

Notes: Kill the King was originally developed for the Stratega framework [43]. There is only a single pre-implemented Kill the King map.

F. NUMBERS RACE

Description: In Numbers Race, players take turns choosing an integer between 1 and $n \in \mathbb{N}$. The goal is to choose a number $m \leq n$ such that the sum of all previous numbers is equal to some goal number $g \in \mathbb{N}$. If this sum exceeds g , then the player who overshot loses.

Action space: At each turn, one has to select an integer $i \in \{1, \dots, n\}$.

Terminal states: The game ends when $i_1 + \dots + i_k \geq g$. In case of equality, the player who chose action i_k wins, and otherwise, that player loses.

Initial states: The sum starts at 0.

Notes: Numbers race was developed by us.

G. SmallOthello

Description: SmallOthello is played on an $n \times n$ board where players take turns placing a stone of their color on an empty cell. Once placed, all opponents' stones are flipped that are contained in a vertical, horizontal or diagonal line that starts at the placed stone and that ends at the first stone of the same color as the placed stone, going in the line's direction. When neither player has an available move, the player with the most stones wins. The classical Othello game is played on an 8×8 board; however, to reduce the complexity, we only consider a 6×6 board, hence the name SmallOthello.

Action space: The player whose turn it is, may place one of its stones at any empty cell that borders at least one enemy stone and whose placement would flip at least one enemy stone as described above.

Terminal states: The game ends when both players have no legal actions to play.

Initial states: The board initially contains a square of four stones in the middle of the board. The top left and bottom right of this square are stones of one color and the remaining two are of the opposite color.

Notes: Othello is also known as Reversi, is a well-known, competitive board game that is played in the real world. Fig. 23 visualizes an Othello state transition.

H. PUSHER

Description: Pusher is played on a 2-dimensional grid. Each player controls an initially equal number of units. The goal is to eliminate all enemy units, which can be achieved by pushing them into holes which are spread around the map. Per step, a player may move one unit to an empty neighboring tile (4-neighborhood) and optionally push an enemy unit in the movement direction by 1 tile if the enemy unit is located at a neighboring tile.

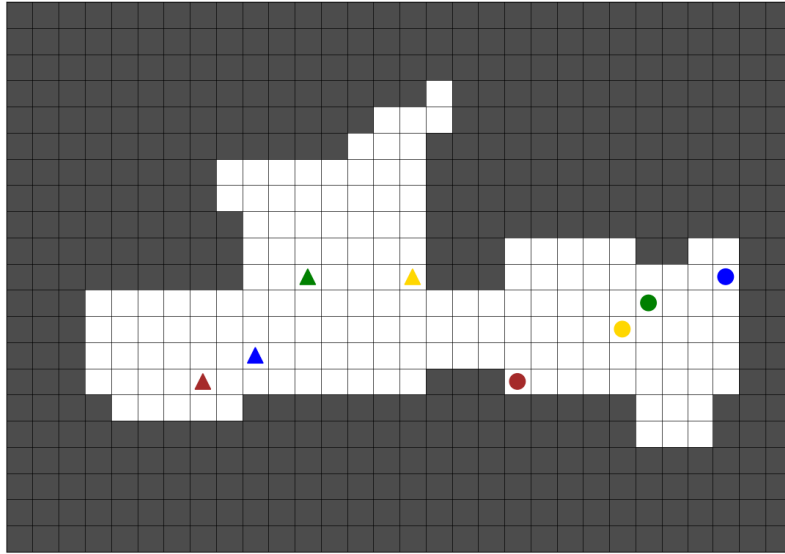
Action space: Per step, the player at turn controls only a single unit. However, the turn only changes if a player has

TABLE 51. Triangle tireworld parameters.

Parameter	Description
Idle	Whether the idle action is allowed.
Reduced action space	Allow only move actions that target a neighboring vertex

TABLE 52. Triangle Tireworld model statistics for the reduced action space setting and idling not being allowed.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	94.32	48.82	3.77	1.25	23.34	110
(2_IPPC)	93.76	37.77	3.80	1.21	27.00	110
(3_IPPC)	89.96	4.25	3.88	1.17	38.66	2647
(4_IPPC)	88.79	-11.70	3.91	1.13	42.64	2619
(5_IPPC)	84.42	-17.73	3.95	1.15	44.68	21882
(6_IPPC)	82.66	-32.84	3.98	1.11	47.44	19128
(7_IPPC)	N/A	-27.81	3.98	1.15	47.12	51574
(8_IPPC)	N/A	-41.13	4.01	1.11	48.97	39305
(9_IPPC)	N/A	-32.31	3.98	1.17	48.19	88707
(10_IPPC)	N/A	-44.57	4.02	1.12	49.50	57505

**FIGURE 22.** An initial unit position of Kill the King. Black squares indicate walls. Green shapes represent healers, blue represents archers, brown represents warriors, and yellow represents kings. Shapes of the same shape belong to the same player. This map is called *standard* in our implementation and was taken from https://github.com/egg-west/Stratega/blob/master/resources/gameConfigurations/TBS/KtKMaps_kwah.yaml.**TABLE 53.** Triangle Tireworld abstraction statistics for the reduced action space setting and idling not being allowed.

Instance	$V_{\text{abs}}(0)$	$Q_{\text{abs}}(0)$	$V_{\text{abs}}(1)$	$Q_{\text{abs}}(1)$	$V_{\text{abs}}(2)$	$Q_{\text{abs}}(2)$
(1_IPPC)	0.11	0.50	0.08	0.40	0.06	0.41
(2_IPPC)	0.09	0.55	0.06	0.46	0.04	0.47
(3_IPPC)	0.05	0.62	0.04	0.56	0.04	0.54
(4_IPPC)	0.04	0.70	0.03	0.64	0.03	0.63
(5_IPPC)	0.03	0.67	0.03	0.61	0.03	0.59
(6_IPPC)	0.02	0.74	0.02	0.69	0.03	0.68

TABLE 54. Wildfire parameters.

Parameter	Description
File	Path to the model instance to be used.

performed an action for all of their alive units. Each unit can idle or be moved to any of the 4 neighboring tiles if they

are not holes or out of bounds. Additionally, these movement actions can be chained with a push action that targets exactly one enemy unit that is within 1 tile of the position after the movement.

TABLE 55. Wildfire model statistics.

Instance	V^*	V_{rndm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	-5701.69	19.00	13.47	50.00	32257
(2_IPPC)	N/A	-18035.25	19.00	5.98	50.00	18897
(3_IPPC)	N/A	-14174.35	33.00	N/A	50.00	1125594
(4_IPPC)	N/A	-29546.28	33.00	N/A	50.00	799121
(5_IPPC)	N/A	-13217.49	51.00	N/A	50.00	2493343
(6_IPPC)	N/A	-36500.43	51.00	N/A	50.00	2491622
(7_IPPC)	N/A	-20330.72	61.00	N/A	50.00	2763448
(8_IPPC)	N/A	-37642.62	61.00	N/A	50.00	2825447
(9_IPPC)	N/A	-23553.28	73.00	N/A	50.00	2922169
(10_IPPC)	N/A	-38359.54	73.00	N/A	50.00	3161935

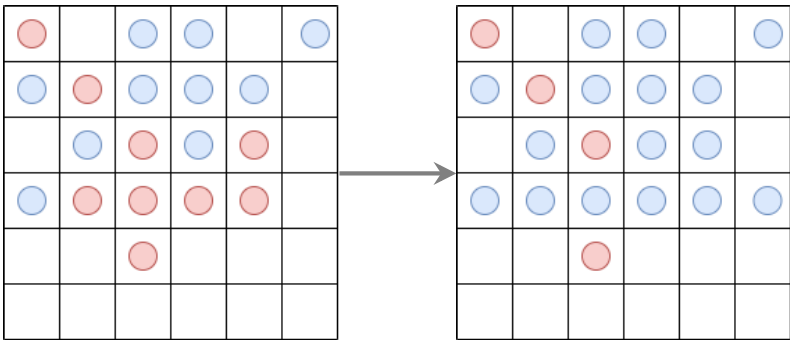


FIGURE 23. An example of a state transition for SmallOthello. In this example, black placed a stone, which caused five white stones to switch their color to black.

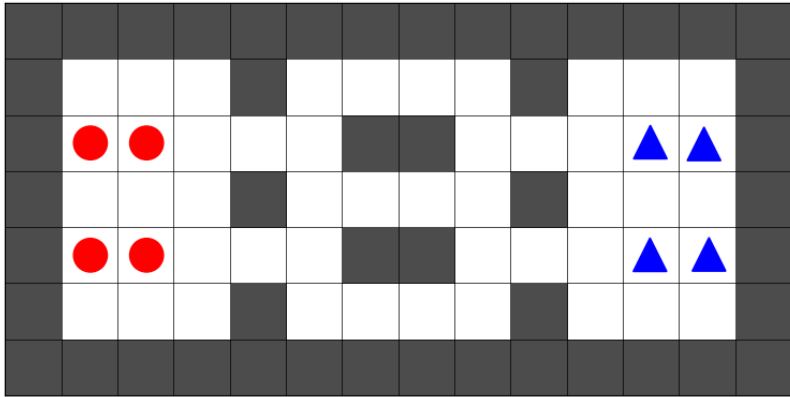


FIGURE 24. The initial unit position of Pusher. Black squares indicate holes. Circles and triangles represent units.

TABLE 56. Wildlife preserve parameters.

Parameter	Description
File	Path to the model instance to be used.

Terminal states: The game ends when either player has no alive units left. The player with no alive units loses. A draw is not possible.

Initial states: The initial configuration is visualized in Fig. 24.

Notes: Pusher was originally developed for the Stratega framework [43]. There is only a single pre-implemented Pusher map.

I. PYLOS

Description: Pylos is played on an initially empty 4 × 4 × 4-dimensional grid on which players take turns placing stones of their own color. Each player starts with 15 stones, which can be placed at any empty cell that is either at the bottom or contains exactly 4 stones in the layer beneath it (i.e., the stone requires a foundation). Instead of placing one of one’s remaining stones, one may also move a stone

TABLE 57. Wildlife preserve model statistics.

Instance	V^*	V_{rdm}	Avg actions	Avg outcomes	Avg episode length	States
(1_IPPC)	N/A	1647.91	4.00	3.20	50.00	62
(2_IPPC)	N/A	1306.04	4.00	3.54	50.00	482
(3_IPPC)	N/A	1319.81	5.00	2.90	50.00	141
(4_IPPC)	N/A	1081.91	25.00	9.51	50.00	1687
(5_IPPC)	N/A	900.49	6.00	5.36	50.00	406
(6_IPPC)	N/A	1160.29	36.00	13.19	50.00	5122
(7_IPPC)	N/A	501.05	7.00	6.96	50.00	2429
(8_IPPC)	N/A	1221.70	49.00	13.87	50.00	16046
(9_IPPC)	N/A	1280.72	8.00	23.92	50.00	4848
(10_IPPC)	N/A	989.97	64.00	65.12	50.00	59767
(11_IPPC)	N/A	1662.97	9.00	27.65	50.00	34985
(12_IPPC)	N/A	913.83	81.00	66.32	50.00	294764
(13_IPPC)	N/A	1302.97	100.00	16.39	50.00	91550
(14_IPPC)	N/A	1226.42	1000.00	56.88	50.00	347464
(15_IPPC)	N/A	2268.32	144.00	70.83	50.00	442714
(16_IPPC)	N/A	1141.27	1728.00	56.03	50.00	590490
(17_IPPC)	N/A	1868.15	2744.00	57.65	50.00	738505
(18_IPPC)	N/A	705.72	2744.00	57.76	50.00	928518
(19_IPPC)	N/A	1985.33	65536.00	54.67	50.00	765431
(20_IPPC)	N/A	1663.98	65536.00	56.05	50.00	951885

TABLE 58. Capture the flag parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 59. Capture the Flag statistics. The statistics are gathered for the zero-sum reward case. The episode length reaches 200 as random agents almost never manage to lose or win a game.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	0.00	10.04	200.00	19784145

TABLE 60. SimpleChess parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 61. Capture the Flag statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	0.00	33.53	60.02	5850833

TABLE 62. Connect 4 parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

of one's color any number of layers upward as long as they are not part of the foundation for another stone. The game ends when one player runs out of stones or when one player completes the pyramid (the bottom layer is of

TABLE 63. Connect 4 statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	0.11	6.76	10.94	760645

TABLE 64. Constrictor parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.
$n \in \mathbb{N}$	The side length of the square grid.

TABLE 65. Constrictor statistics. The statistics are gathered for the zero-sum reward case. The instance column indicates the arena size n .

Instance	V_{rdm}	Avg actions	States
(10)	0.01	2.29	17.68
(15)	-0.00	2.39	22.66
(20)	-0.01	2.43	24.82

the dimension 4×4 , then 3×3 , 2×2 , and finally 1×1).

Action space: The player at turn must place one of its stones either at an empty bottom layer cell or a cell that is supported by 4 stones directly beneath it. The player has to choose between using one of the stones in its stack or moving one of its stones that are already on the board that do not support any stones above it.

TABLE 66. A list of unit properties. A unit's ability can target all units that are within the ability range distance (maximum x and y distance must be less than this range) and damages/heals the target for the ability amount. Only enemy units can be attacked, and only friendly units can be healed.

Unit	Health	Ability range	Ability amount	Ability type
King	200	1	50	Attack
Warrior	100	1	70	Attack
Archer	100	2	50	Attack
Healer	50	2	50	Heal

TABLE 67. Kill the king parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 68. Kill the king statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	-0.07	6.37	152.30	14940064

TABLE 69. Numbers race parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.
$n \in \mathbb{N}$	The maximum number a player may choose.
$g \in \mathbb{N}$	The goal number.

TABLE 70. Numbers race statistics. The statistics are gathered for the zero-sum reward case. The instance format is (g, n) .

Instance	V_{rdm}	Avg actions	Avg episode length	States
(20,5)	0.01	5.00	3.80	28
(20,10)	-0.01	10.00	2.35	38
(100,5)	-0.00	5.00	17.14	108
(100,10)	-0.00	10.00	9.61	118

TABLE 71. SmallOthello parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 72. SmallOthello statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	-0.12	5.47	15.57	1227758

TABLE 73. Pusher parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

Terminal states: The game ends when one player runs out of stones. The player who runs out of stones loses.

TABLE 74. Pusher statistics. The statistics are gathered for the zero-sum reward case. The average episode length is close to 200, as random agents almost never manage to lose or win a game.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	-0.01	3.67	199.75	18203006

TABLE 75. Pylos parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 76. Pylos. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	-0.23	7.47	15.94	1370885

TABLE 77. Quarto parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 78. Quarto statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	0.01	10.41	11.66	1017474

TABLE 79. Tic Tac Toe parameters.

Parameter	Description
Zero-sum	Whether the rewards are zero-sum or binary 1 for win, 0 otherwise.

TABLE 80. Tic Tac Toe statistics. The statistics are gathered for the zero-sum reward case.

Instance	V_{rdm}	Avg actions	Avg episode length	States
(standard)	0.30	5.71	4.17	3380

A player may also win if they place a stone at the 4th layer, i.e., completes the pyramid. There are no draws in Pylos.

Initial states: The board is initially empty.

Notes: There are several versions of Pylos that include piece recovering when one completes a square of one's own color. We did not implement these versions. Pylos is a lesser-known real-world board game invented by David G. Royffe. Fig. 25 shows a real-world game of Pylos in progress.²

²https://commons.wikimedia.org/wiki/File:Pylos_2.jpg by Matěj Baha, 2007, CC BY-SA 2.5 via Wikimedia Commons.

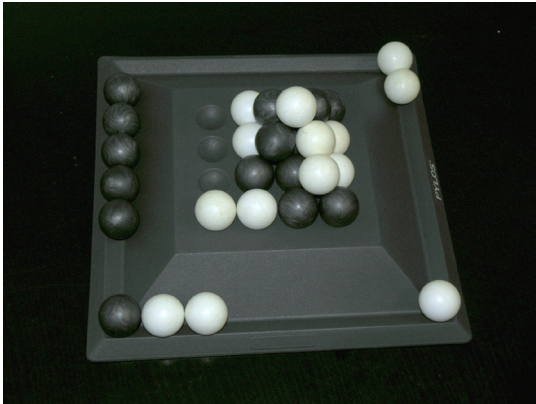


FIGURE 25. A real-life game of Pylos in progress.



FIGURE 26. A real-life empty Quarto board. On the side, all 16 unique tiles can be seen.

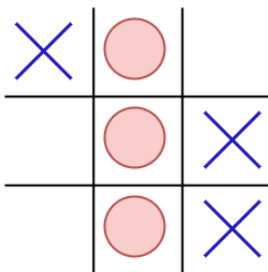


FIGURE 27. Visualization of a Tic Tac Toe game state in which the player with the red circles won because a vertical row was completed.

J. QUARTO

Description: Quarto is played on a 4×4 grid. The goal is to complete a vertical, horizontal, or diagonal line of length 4 with stones that all share a common property. There are initially 16 stones that can be placed on the board. The set of stones is given by $S = \{0, 1\}^4$ where the i -th component of a stone is referred to as a property. Players take turns

placing a stone and then choosing one of the remaining, not yet placed stones, which the opponent must place in the next turn (the game starts with one player selecting a stone for the opponent).

Action space: There are two types of turns. The placement turn, where the player must place the stone chosen in the previous step by the opponent into one of the empty cells. In the stone-selection turn, the player must choose one of the not-yet-placed stones that the opponent must use in the next turn.

Terminal states: The game ends when one player manages to complete a vertical, horizontal, or diagonal line of size 4 with stones that all share a common property. In Quarto, there is no concept of stone ownership; hence, the line may be completed with stones that have previously been laid down by the opponent.

Initial states: The board is initially empty. The first turn is a select-enemy-stone move.

Notes: The four properties are referred to as its size, brightness, shape, and whether they contain a hole or not. Quarto was invented by Blaise Müller and is a lesser-known real-world board game. Fig. 26 shows a real-world Quarto board and its tiles.³

K. TIC TAC TOE

Description: Tic Tac Toe is played on a grid of width and height 3. Each player is assigned a color and places one stone of their color in one of the empty grid cells. The first player to create a vertical, horizontal, or diagonal row of three same-colored stones wins.

Action space: Place a stone of one's own color in an empty grid cell.

Terminal states: The game ends if one player manages to complete a horizontal, vertical, or diagonal line of three of their stones. If a stone is placed in all 9 grid cells but no player won, the game ends in a draw.

Initial states: The board is initially empty.

Notes: Tic Tac Toe is a well-known board game that is small enough to be fully solved with MiniMax search. If both players play perfectly, no one can force a win. Fig. 27 shows a Tic Tac Toe game state.

REFERENCES

- [1] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012, doi: [10.1109/TCI-AIG.2012.2186810](https://doi.org/10.1109/TCI-AIG.2012.2186810).
- [2] R. Schmöcker, L. Kampmann, and A. Dockhorn, "Time-critical and confidence-based abstraction dropping methods," in *Proc. IEEE Conf. Games (CoG)*, Aug. 2025, pp. 1–8.
- [3] A. Dockhorn, J. Hurtado-Grueso, D. Jeurissen, L. Xu, and D. Perez-Liebana, "Game state and action abstracting Monte Carlo tree search for general strategy game-playing," in *Proc. IEEE Conf. Games (CoG)*, Aug. 2021, pp. 1–8.

³<https://commons.wikimedia.org/wiki/File:QuartoSpiel.JPG> by Immanuel Giel, 2005, CC BY-SA 3.0 via Wikimedia Commons.

- [4] A. Anand, A. Grover, M. Mausam, and P. Singla, "ASAP-UCT: Abstraction of state-action pairs in UCT," in *Proc. 24th Int. Joint Conf. Artif. Intell. (IJCAI)*, Buenos Aires, Argentina, 2015, pp. 1509–1515. [Online]. Available: <http://ijcai.org/Abstract/15/216>
- [5] A. Anand, R. Noothigattu, M. Mausam, and P. Singla, "OGA-UCT: On-the-go abstractions in UCT," in *Proc. 26th Int. Conf. Automated Planning Scheduling*, vol. 26, 2016, pp. 29–37.
- [6] N. Jiang, S. Singh, and R. L. Lewis, "Improving UCT planning via approximate homomorphisms," in *Proc. Int. Conf. Auto. Agents Multi-Agent Syst. (AAMAS)*, Paris, France, A. L. C. Bazzan, M. N. Huhns, A. Lomuscio, and P. Scerri, Eds., 2014, pp. 1289–1296. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2617453>
- [7] S. Sanner. (2010). *Relational Dynamic Influence Diagram Language (RDDL): Language Description*. [Online]. Available: <http://users.cecs.anu.edu.au/>
- [8] D. Perez-Liebana, A. Dockhorn, J. H. Grueso, and D. Jeurissen, "The design of 'Stratega': A general strategy games framework," 2020, *arXiv:2009.05643*.
- [9] J. Hostetler, A. Fern, and T. G. Dietterich, "Progressive abstraction refinement for sparse sampling," in *Proc. 21st Conf. Uncertainty Artif. Intell. (UAI)*, 2015, pp. 365–374. [Online]. Available: <http://auai.org/uai2015/proceedings/papers/81.pdf>
- [10] L. Xu, A. Dockhorn, and D. Perez-Liebana, "Elastic Monte Carlo tree search," *IEEE Trans. Games*, vol. 15, no. 4, pp. 527–537, Dec. 2023.
- [11] A. Taitler, M. Gimelfarb, J. Jeong, S. Gopalakrishnan, M. Mladenov, X. Liu, and S. Sanner, "PyRDDLGym: From RDDL to gym environments," 2022, *arXiv:2211.05939*.
- [12] S. Yoon, A. Fern, R. Givan, and S. Kambhampati, "Probabilistic planning via determinization in hindsight," in *Proc. 23rd AAAI Conf. Artif. Intell.*, Jul. 2008, pp. 1010–1016. [Online]. Available: <http://www.aaai.org/Library/AAAI/2008/aaai08-160.php>
- [13] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," 2016, *arXiv:1606.01540*.
- [14] É. Piette, M. Stephenson, D. J. N. J. Soemers, and C. Browne, "General board game concepts," in *Proc. IEEE Conf. Games (CoG)*, Copenhagen, Denmark, Aug. 2021, pp. 01–08, doi: [10.1109/COG52621.2021.9618990](https://doi.org/10.1109/COG52621.2021.9618990).
- [15] É. Piette, D. J. N. J. Soemers, M. Stephenson, C. F. Sironi, M. H. M. Winands, and C. Browne, "Ludii - the ludemic general game system," in *Proc. 24th Europ. Conf. Artif. Intell. (ECAI)*, 2022, pp. 411–418.
- [16] C. Bamford, "Griddly: A platform for AI research in games," *Softw. Impacts*, vol. 8, May 2021, Art. no. 100066. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2665963821000142>
- [17] B. Bonet and H. Geffner. (2012). *MDP-Engine*. [Online]. Available: <https://github.com/bonetblai/mdp-engine>
- [18] M. Lanctot et al., "OpenSpiel: A framework for reinforcement learning in games," 2019, *arXiv:1908.09453*.
- [19] S. Koyamada, S. Okano, S. Nishimori, Y. Murata, K. Habara, H. Kita, and S. Ishii, "PgX: Hardware-accelerated parallel game simulators for reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 45716–45743.
- [20] M. Chevalier-Boisvert, L. Willems, and S. Pal. (2018). *Minimalistic Gridworld Environment for OpenAI Gym*. [Online]. Available: <https://github.com/maximecb/gym-minigrd>
- [21] D. Perez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game AI: A multitrack framework for evaluating agents, games, and content generation algorithms," *IEEE Trans. Games*, vol. 11, no. 3, pp. 195–214, Sep. 2019, doi: [10.1109/TG.2019.2901021](https://doi.org/10.1109/TG.2019.2901021).
- [22] C. Beattie, T. Köppe, E. A. Duéñez-Guzmán, and J. Z. Leibo, "DeepMind Lab2D," 2020, *arXiv:2011.07027*.
- [23] S. Sanner, S. Yoon, and T. Walsh. *ICAPS 2011 International Probabilistic Planning Competition (IPPC)*. Accessed: Sep. 16, 2025. [Online]. Available: <https://users.cecs.anu.edu.au/ssanner/IPPC2011/>
- [24] J. T. Guerin, J. P. Hanna, L. Ferland, N. Mattei, and J. Goldsmith, "The academic advising planning domain," presented at the 3rd Workshop Int. Planning Competition, Sao Paulo, Brazil, 2012.
- [25] M. Grzes, J. Hoey, and S. Sanner. *ICAPS 2014 International Probabilistic Planning Competition (IPPC) Discrete Track*. Accessed: Sep. 16, 2025. [Online]. Available: <https://www.cs.kent.ac.uk/people/staff/mg483/IPPC2014/>
- [26] A. Hertle, C. Dornhege, T. Keller, R. Mattmüller, M. Ortlieb, and B. Nebel, "An experimental comparison of classical, FOND and probabilistic planning," in *Proc. 37th Adv. Artif. Intell. Annu. German Conf. Artif. Intell.*, Stuttgart, Germany, Sep. 2014, pp. 297–308, doi: [10.1007/978-3-319-11206-0_29](https://doi.org/10.1007/978-3-319-11206-0_29).
- [27] M. Gardner, "The fantastic combinations of John Conway's new solitaire game 'life,'" *Sci. American*, vol. 223, no. 4, pp. 120–123, Oct. 1970. [Online]. Available: <https://www.jstor.org/stable/24927642>
- [28] C. Boyer. *Morpion Solitaire*. Accessed: Jul. 3, 2025. [Online]. Available: <http://www.morpionsolitaire.com/>
- [29] M. Gardner, "Mathematical games," *Sci. Amer.*, vol. 223, no. 4, p. 108, Jan. 1973.
- [30] A. G. Barto, S. J. Bradtko, and S. P. Singh, "Learning to act using real-time dynamic programming," *Artif. Intell.*, vol. 72, nos. 1–2, pp. 81–138, Jan. 1995, doi: [10.1016/0004-3702\(94\)00011-o](https://doi.org/10.1016/0004-3702(94)00011-o).
- [31] J. Hostetler, A. Fern, and T. Dietterich, "Sample-based tree search with fixed and adaptive state abstractions," *J. Artif. Intell. Res.*, vol. 60, pp. 717–777, Dec. 2017, doi: [10.1613/jair.5483](https://doi.org/10.1613/jair.5483).
- [32] S. Nicol, R. Sabbadin, N. Peyrard, and I. Chadès, "Finding the best management policy to eradicate invasive species from spatial ecological networks with simultaneous actions," *J. Appl. Ecology*, vol. 54, no. 6, pp. 1989–1999, Dec. 2017. [Online]. Available: <https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/1365-2664.12884>
- [33] R. Vanderbei, "Optimal sailing strategies," Dept. Oper. Res. Financial Eng. (Wikipedia), Univ. Princeton, Princeton, NJ, USA, 1996. [Online]. Available: <http://www.sor.princeton.edu/É?rvdb/sail/sail.html>
- [34] C. Guestrin, D. Koller, R. Parr, and S. Venkataraman, "Efficient solution algorithms for factored MDPs," *J. Artif. Intell. Res.*, vol. 19, pp. 399–468, Oct. 2003, doi: [10.1613/jair.1000](https://doi.org/10.1613/jair.1000).
- [35] T. G. Dietterich, M. A. Taleghan, and M. Crowley, "PAC optimal planning for invasive species management: Improved exploration for reinforcement learning from simulator-defined MDPs," in *Proc. 27th AAAI Conf. Artif. Intell.*, vol. 27, Jul. 2013, pp. 1270–1276, doi: [10.1609/aaai.v27i1.8487](https://doi.org/10.1609/aaai.v27i1.8487).
- [36] R. Muneeppeerakul, J. S. Weitz, S. A. Levin, A. Rinaldo, and I. Rodríguez-Iturbe, "A neutral metapopulation model of biodiversity in river networks," *J. Theor. Biol.*, vol. 245, no. 2, pp. 351–363, Mar. 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0022519306004747>
- [37] K. V. Delgado, S. Sanner, and L. N. de Barros, "Efficient solutions to factored MDPs with imprecise transition probabilities," *Artif. Intell.*, vol. 175, nos. 9–10, pp. 1498–1527, Jun. 2011. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370211000026>
- [38] C. F. Daganzo, "The cell transition model: Network traffic," Berkeley Inst. Transp. Stud., Berkeley, CA, USA, Tech. Rep., 1994.
- [39] I. Little and S. Thiebaux, "Probabilistic planning vs replanning," presented at the ICAPS Workshop Planning Competitions, Past, Present, and Future, 2007.
- [40] I. Karafyllidis and A. Thanailakis, "A model for predicting forest fire spreading using cellular automata," *Ecol. Model.*, vol. 99, no. 1, pp. 87–97, Jun. 1997. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0304380096019424>
- [41] F. Fang, P. Stone, and M. Tambe, "When security games go green: Designing defender strategies to prevent poaching and illegal fishing," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, Buenos Aires, Argentina, Jul. 2015, pp. 2589–2595. [Online]. Available: <http://ijcai.org/Abstract/15/367>
- [42] T. Nguyen, R. Yang, A. Azaria, S. Kraus, and M. Tambe, "Analyzing the effectiveness of adversary modeling in security games," in *Proc. 27th AAAI Conf. Artif. Intell.*, vol. 27, Jul. 2013, pp. 718–724, doi: [10.1609/aaai.v27i1.8599](https://doi.org/10.1609/aaai.v27i1.8599).
- [43] A. Dockhorn, J. H. Grueso, D. Jeurissen, and D. P. Liebana, "STRATEGA: A general strategy games framework," in *Proc. Workshops Co-Located 16th AAAI Conf. Artif. Intell. Interact. Digital (AIIDE)*, vol. 2862, Worcester, MA, USA, 2020, pp. 1–11. [Online]. Available: <https://ceur-ws.org/Vol-2862/paper30.pdf>
- [44] *What is Battlesnake?*. Accessed: Jun. 3, 2025. [Online]. Available: <https://docs.battlesnake.com/>



ROBIN SCHMÖCKER received the Master of Computer Science and Mathematics degrees from Leibniz University Hannover, Germany, in 2023 and 2024, respectively, where he is currently pursuing the Ph.D. degree in artificial intelligence with the Institute of Information Processing. His research interests include abstraction methods for Monte-Carlo-based tree searches and ensemble techniques for reinforcement learning.



BODO ROSENHAHN received the degree in computer science (minor subject medicine) from the University of Kiel and the Dipl.-Inf. and Dr.-Ing. degrees from the University of Kiel, in 1999 and 2003, respectively. From 2003 to 2005, he was a Postdoctoral Researcher at The University of Auckland, New Zealand, funded with a scholarship from German Research Foundation (DFG). From 2005 to 2008, he was a Senior Researcher at the Max-Planck Institute for Informatics in Saarbruecken. Since 2008, he has been a Full Professor at Leibniz University Hannover, where he is heading a group on automated image interpretation.

...



ALEXANDER DOCKHORN (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in computer science from Otto von Guericke University Magdeburg (OVGU). His postdoctoral work spans the Queen Mary University of London and OVGU, followed by a Junior Professorship at Leibniz University Hannover. He is currently an Associate Professor with the Metaverse Laboratory, University of Southern Denmark (SDU), Odense, where he works on artificial intelligence

in games and its transfer to broader applications. Visit his website for publications and project details: <https://adockhorn.github.io/>